



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

گزارش آزمایش

گزارش آزمایش اول

طراحی مدار ترکیبی تشخیص بخش پذیری عدد BCD چهاررقمی

اعضای گروه

کیارش شجاعی آرانی

سینا فعالیت

پارسا ملکیان

نام درس

آزمایشگاه طراحی سامانه‌های رقمی (گروه یکشنبه)

نیم‌سال تابستان ۱۴۰۴-۱۴۰۵

نام استاد درس

دکتر اجلالی

دستیار آموزشی

علی اثنی‌عشری

فهرست مطالب

۱	مقدمه	۳
۱-۱	صورت مسئله	۳
۱-۲	یادآوری کد BCD	۴
۲	بخش اول: تشخیص بخش پذیری بر ۳	۴
۲-۱	ایده کلی و اثبات ریاضی	۴
۲-۲	کاهش پیچیدگی سخت افزاری	۴
۲-۲-۱	پیمانه MOD_3 (تبدیل رقم BCD به باقی مانده)	۴
۲-۲-۲	پیمانه FAX2_MOD_3 (جمع کننده و باقی مانده گیر ادغامی)	۵
۲-۳	ساختار کلی مدار	۵
۲-۴	طراحی پیمانه FAX2_MOD_3	۵
۲-۵	طراحی پیمانه MOD_3	۷
۲-۶	مدار اصلی	۹
۲-۷	نتایج تست ها	۱۰
۲-۷-۱	تست پیمانه FAX2_MOD_3	۱۰
۲-۷-۱-۱	تست ۲ + ۲	۱۰
۲-۷-۱-۲	تست ۱ + ۱	۱۱
۲-۷-۱-۳	تست ۲ + ۱	۱۲
۲-۷-۲	تست پیمانه MOD_3	۱۲
۲-۷-۲-۱	تست ۱	۱۲
۲-۷-۲-۲	تست ۴	۱۳
۲-۷-۲-۳	تست ۵	۱۳
۲-۷-۲-۴	تست ۹	۱۳
۲-۷-۳	تست مدار کلی	۱۴
۲-۷-۳-۱	تست ۰۰۰۰	۱۴
۲-۷-۳-۲	تست ۰۰۹۹	۱۴
۲-۷-۳-۳	تست ۰۵۷۶	۱۴
۲-۷-۳-۴	تست ۳۵۷۶	۱۵
۲-۷-۳-۵	تست ۹۵۷۶	۱۵
۲-۷-۳-۶	تست ۸۰۱۱	۱۶
۳	بخش دوم: تشخیص بخش پذیری بر ۱۱	۱۶
۳-۱	ایده کلی و اثبات ریاضی	۱۶
۳-۲	ساختار کلی مدار	۱۷
۳-۳	طراحی پیمانه MOD_11_NEG	۱۷
۳-۴	طراحی پیمانه FAX4_MOD_11	۱۹
۳-۵	مدار اصلی	۲۱

۲۱	نتایج تست‌ها	۳-۶
۲۱	تست پیمانه MOD_11_NEG	۳-۶-۱
۲۱	تست ۰	۳-۶-۱-۱
۲۲	تست ۳	۳-۶-۱-۲
۲۲	تست ۹	۳-۶-۱-۳
۲۳	تست پیمانه FAX4	۳-۶-۲
۲۳	تست ۹ + ۹	۳-۶-۲-۱
۲۴	تست پیمانه FAX4_MOD_11	۳-۶-۳
۲۴	تست ۰ + ۵	۳-۶-۳-۱
۲۴	تست ۹ + ۵	۳-۶-۳-۲
۲۵	تست ۱۰ + ۹	۳-۶-۳-۳
۲۵	تست ۰ + ۱۰	۳-۶-۳-۴
۲۶	تست ۳ + ۷	۳-۶-۳-۵
۲۶	تست ۷ + ۴	۳-۶-۳-۶
۲۷	تست مدار کلی	۳-۶-۴
۲۷	تست ۰۰۰۰	۳-۶-۴-۱
۲۸	تست ۰۰۰۱	۳-۶-۴-۲
۲۸	تست ۰۰۱۱	۳-۶-۴-۳
۲۹	تست ۰۰۹۹	۳-۶-۴-۴
۲۹	تست ۹۲۷۳	۳-۶-۴-۵
۳۰	تست ۹۲۷۴	۳-۶-۴-۶
۳۰	تست ۰۹۰۲	۳-۶-۴-۷
۳۱	جمع‌بندی	۴

چکیده: در این آزمایش دو مدار ترکیبی طراحی و در نرم افزار Quartus پیاده سازی و شبیه سازی شده اند که یک عدد چهاررقمی در کد BCD را در ورودی می گیرند و در خروجی تک بیتی خود مشخص می کنند که آیا آن عدد بر ۳ (و در بخش دوم، بر ۱۱) بخش پذیر است یا نه. بنا بر صورت آزمایش، در طراحی این مدارها تنها استفاده از دروازه های پایه AND، OR و NOT مجاز بوده است؛ بنابراین کل طراحی به صورت طرح وارده^۱ و بدون بهره گیری از بلوک های آماده کتابخانه ای انجام شده است. ایده ی اصلی طراحی، استفاده از خواص باقی مانده ی توان های ۱۰ به پیمانه ی ۳ و ۱۱ است: از آنجا که $10 \equiv 1 \pmod{3}$ ، بخش پذیری بر ۳ به جمع باقی مانده ی ارقام به پیمانه ی ۳ تبدیل می شود و از آنجا که $10 \equiv -1 \pmod{11}$ ، بخش پذیری بر ۱۱ به بررسی تفاضل متناوب ارقام کاهش می یابد. این کاهش ها باعث می شوند به جای کار با یک عدد ۱۶ بیتی، تنها با مدارهای کوچک دوبیتی (در بخش اول) و چهاربیتی (در بخش دوم) سروکار داشته باشیم.

۱ مقدمه

شرکت های تولیدکننده ی تراشه های FPGA و CPLD علاوه بر تولید خود تراشه، نرم افزارهایی نیز به عنوان ابزار طراحی به کمک کامپیوتر^۲ در اختیار کاربران قرار می دهند. این ابزارها تنها به زبان های استاندارد توصیف سخت افزار مانند Verilog و VHDL محدود نیستند؛ بلکه برای طراحی سریع مدارهای کوچک، امکان طرح وارده را نیز فراهم می کنند که در آن طراح مستقیماً دروازه ها و اتصال های میان آن ها را رسم می کند. هدف آزمایش اول آشنایی عملی با این امکان و طراحی و پیاده سازی یک مدار ترکیبی^۳ با استفاده از آن است.

۱-۱ صورت مسئله

مداری ترکیبی طراحی کنید که یک ورودی چهاررقمی^۴ BCD و یک خروجی یک بیتی داشته باشد. خروجی مدار تنها زمانی برابر یک شود که عدد ورودی مضربی از ۳ باشد. سپس همین آزمایش را برای اعداد مضرب ۱۱ نیز تکرار کنید. در طراحی این مدار تنها استفاده از دروازه های پایه، یعنی AND، OR و NOT، مجاز است.

بنابراین مدار مورد نظر دارای ۱۶ خط ورودی و یک خط خروجی است. در سراسر این گزارش، چهار رقم عدد ورودی را از کم ارزش به پر ارزش با A (یکان)، B (دهگان)، C (صدگان) و D (هزارگان) نشان می دهیم، که همان نام گذاری پایه های ورودی در طرح وارده های Quartus نیز هست. هر رقم با چهار بیت کد می شود؛ برای نمونه رقم A با بیت های A0 A1 A2 A3 که در آن کم ارزش ترین بیت است. خروجی تک بیتی مدار را با Y نشان می دهیم.

دقت کنید که در کد BCD تنها ترکیب های ۰۰۰۰ تا ۱۰۰۱ معنادار هستند و شش ترکیب ۱۰۱۰ تا ۱۱۱۱ هرگز به عنوان ورودی معتبر ظاهر نمی شوند. این شش حالت در طراحی به عنوان بی اهمیت^۵ در نظر گرفته می شوند و در ساده سازی نقشه های کارنو^۶ از آن ها استفاده می کنیم.

^۱ Schematic
^۲ Computer Aided Design (CAD)
^۳ Combinational Logic
^۴ Binary Coded Decimal
^۵ Don't Care
^۶ Karnaugh Map
^۷ Binary

۱-۲ یادآوری کد BCD

در کد BCD هر رقم کد دهدهی (۰ تا ۹) به صورت یک عدد دودویی^۷ چهاربیتی نمایش داده می شود. در نتیجه عدد صفر در کد دهدهی به صورت ۰۰۰۰ و عدد نه به صورت ۱۰۰۱ خواهد بود. جدول ۱ این تبدیل را به طور کامل نشان می دهد. از جمله کاربردهای این کد می توان به ساعت ها و دماسنج های دیجیتال اشاره کرد، که در آن ها هر رقم مستقیماً به یک نمایشگر هفت قسمتی^۸ متصل می شود و بنابراین نگهداری عدد به صورت رقم به رقم بسیار ساده تر از نگهداری آن به صورت دودویی خالص است.

جدول ۱: تبدیل کد دهدهی به BCD

دهدهی	۰	۱	۲	۳	۴	۵	۶	۷	۸	۹
BCD	۰۰۰۰	۰۰۰۱	۰۰۱۰	۰۰۱۱	۰۱۰۰	۰۱۰۱	۰۱۱۰	۰۱۱۱	۱۰۰۰	۱۰۰۱

۲ بخش اول: تشخیص بخش پذیری بر ۳

۲-۱ ایده کلی و اثبات ریاضی

برای بررسی بخش پذیری یک عدد بر ۳ در مبنای ۱۰، از ویژگی باقی مانده استفاده می کنیم. هر عدد ۴ رقمی N با ارقام D, C, B, A را می توان به صورت زیر نوشت:

$$(۱) \quad N = D \times ۱۰^۳ + C \times ۱۰^۲ + B \times ۱۰^۱ + A$$

لم ۱. برای عدد N با ارقام دهدهی $DCBA$ داریم $N \equiv A + B + C + D \pmod{۳}$.

اثبات. از آنجا که ۱۰ در پیمانه ۳ باقی مانده ۱ دارد ($۱۰ \equiv ۱ \pmod{۳}$)، تمام توان های ۱۰ نیز در پیمانه ۳ برابر ۱ خواهند بود ($۱۰^k \equiv ۱ \pmod{۳}$). بنابراین

$$(۲) \quad N \equiv D \times ۱ + C \times ۱ + B \times ۱ + A \equiv A + B + C + D \pmod{۳}$$

□

این یعنی کافی است بررسی کنیم آیا جمع تمام ارقام بر ۳ بخش پذیر است یا خیر.

۲-۲ کاهش پیچیدگی سخت افزاری

اگر بخواهیم مستقیم ۴ رقم ۴-بیتی BCD را با هم جمع کنیم، به جمع کننده های چندبیتی و مدارهای بزرگ تری نیاز داریم. برای اینکه طراحی ساده تر بماند، روند محاسبه را در دو پیمانه ی منطقی ساده خرد و ترکیب می کنیم تا تعداد بیت های محاسباتی در تمام مراحل حداکثر ۲ بیت باقی بماند.

۲-۲-۱ پیمانه MOD_3 (تبدیل رقم BCD به باقی مانده)

در مرحله اول، به جای ورودی ۴-بیتی هر رقم BCD (۰ تا ۹)، مستقیم باقی مانده آن بر ۳ را محاسبه می کنیم. چون باقی مانده تقسیم بر ۳ همیشه یکی از اعداد ۰، ۱ یا ۲ است، خروجی این مرحله فقط ۲ بیت خواهد بود:

- ارقام ۰، ۳، ۶ و ۹ باقی مانده ی ۰ دارند، یعنی کد ۰۰
- ارقام ۱، ۴ و ۷ باقی مانده ی ۱ دارند، یعنی کد ۰۱
- ارقام ۲، ۵ و ۸ باقی مانده ی ۲ دارند، یعنی کد ۱۰

^۸Seven Segment Display

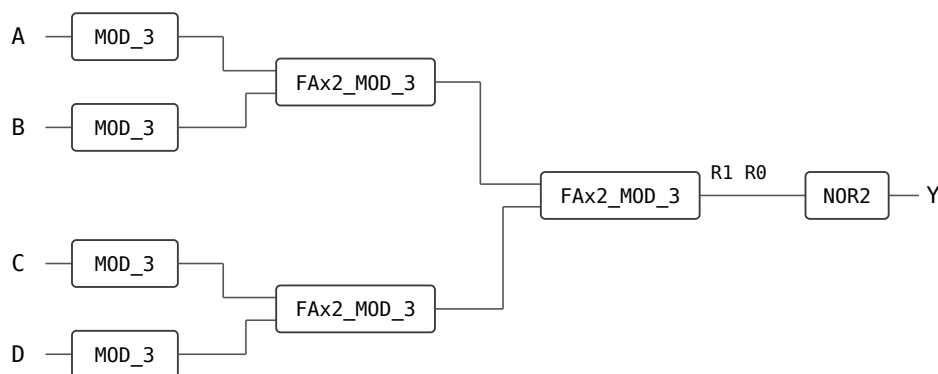
نام پورت‌های این پیمانه در طرح‌واره به این صورت است: ورودی $A_3 A_2 A_1 A_0$ (رقم BCD) و خروجی $R_1 R_0$ (باقی‌مانده بر ۳).

۲-۲-۲ پیمانه $Fax2_MOD_3$ (جمع‌کننده و باقی‌مانده‌گیر ادغامی)

به جای اینکه دو عدد را جمع کنیم و بعد یک مدار جداگانه برای باقی‌مانده‌گیری بگذاریم، این دو عملیات را تلفیق می‌کنیم. این پیمانه دو ورودی ۲-بیتی $A_1 A_0$ و $B_1 B_0$ می‌گیرد که هرکدام مقادیر ۰، ۱ یا ۲ هستند، و باقی‌مانده‌ی جمع آن‌ها بر ۳ را در قالب یک خروجی ۲-بیتی $R_1 R_0$ تحویل می‌دهد. همچنین چون این عملیات در سطح ۲ بیت انجام می‌شود، نیاز به یک جمع‌کننده‌ی کامل^۹ نیست و می‌شود کاملاً به صورت مستقیم انجام شود.

۲-۳ ساختار کلی مدار

محاسبات را به صورت درختی و مرحله‌وار انجام می‌دهیم:



شکل ۱: ساختار درختی مدار تشخیص بخش‌پذیری بر ۳

۱. مرحله اول: ۴ رقم BCD هم‌زمان وارد ۴ پیمانه MOD_3 شده و به ۴ باقی‌مانده ۲-بیتی تبدیل می‌شوند.
۲. مرحله دوم: باقی‌مانده‌ها دوباره وارد پیمانه‌های $Fax2_MOD_3$ می‌شوند تا باقی‌مانده‌ی $A + B$ و باقی‌مانده‌ی $C + D$ بر ۳ به دست بیاید.
۳. مرحله سوم: حاصل این دو مرحله وارد آخرین $Fax2_MOD_3$ شده تا باقی‌مانده کل عدد محاسبه شود.
۴. مرحله نهایی (تشخیص صفر): اگر خروجی نهایی $R_1 R_0$ برابر ۰۰ باشد، یعنی باقی‌مانده کل صفر است. این کار تنها با یک دروازه‌ی NOR دو ورودی انجام می‌شود و خروجی ۱ به معنی بخش‌پذیر بودن عدد بر ۳ است.

۲-۴ طراحی پیمانه $Fax2_MOD_3$

ابتدا این پیمانه را می‌سازیم، اینکار به کمک نقشه‌ی کارنو به سادگی انجام می‌شود. جدول درستی آن در جدول ۲ آمده است. باید حواسمان باشد که مقادیر بی‌اهمیت به این دلیل وجود دارند که ورودی خودش باقی‌مانده بر ۳ است و نمی‌تواند ۱۱ یعنی ۳ باشد، پس خروجی بی‌اهمیت می‌شود.

^۹ Full Adder

جدول ۲: جدول درستی پیمانه FAX2_MOD_3

$R_1 R_0$	باقی مانده ی $A + B$ بر ۳	B_0	B_1	B	A_0	A_1	A
۰۰	۰	۰	۰	۰	۰	۰	۰
۰۱	۱	۱	۰	۱	۰	۰	۰
۱۰	۲	۰	۱	۲	۰	۰	۰
۰۱	۱	۰	۰	۰	۱	۰	۱
۱۰	۲	۱	۰	۱	۱	۰	۱
۰۰	۰	۰	۱	۲	۱	۰	۱
۱۰	۲	۰	۰	۰	۰	۱	۲
۰۰	۰	۱	۰	۱	۰	۱	۲
۰۱	۱	۰	۱	۲	۰	۱	۲
× ×	—	—	—	—	۱	۱	—
× ×	—	۱	۱	—	—	—	—

جدول ۳: نقشه ی کارنوی R_0 — سطرها $A_0 A_1$ و ستون ها $B_0 B_1$

R_0	۰۰	۰۱	۱۱	۱۰
۰۰	۰	۱	×	۰
۰۱	۱	۰	×	۰
۱۱	×	×	×	×
۱۰	۰	۰	×	۱

طبق نقشه ی کارنو، فرمول نهایی برای بیت اول به صورت زیر می شود:

$$(۳) \quad R_0 = A_0 \overline{B_1 B_0} + \overline{A_1 A_0} B_0 + A_1 B_1$$

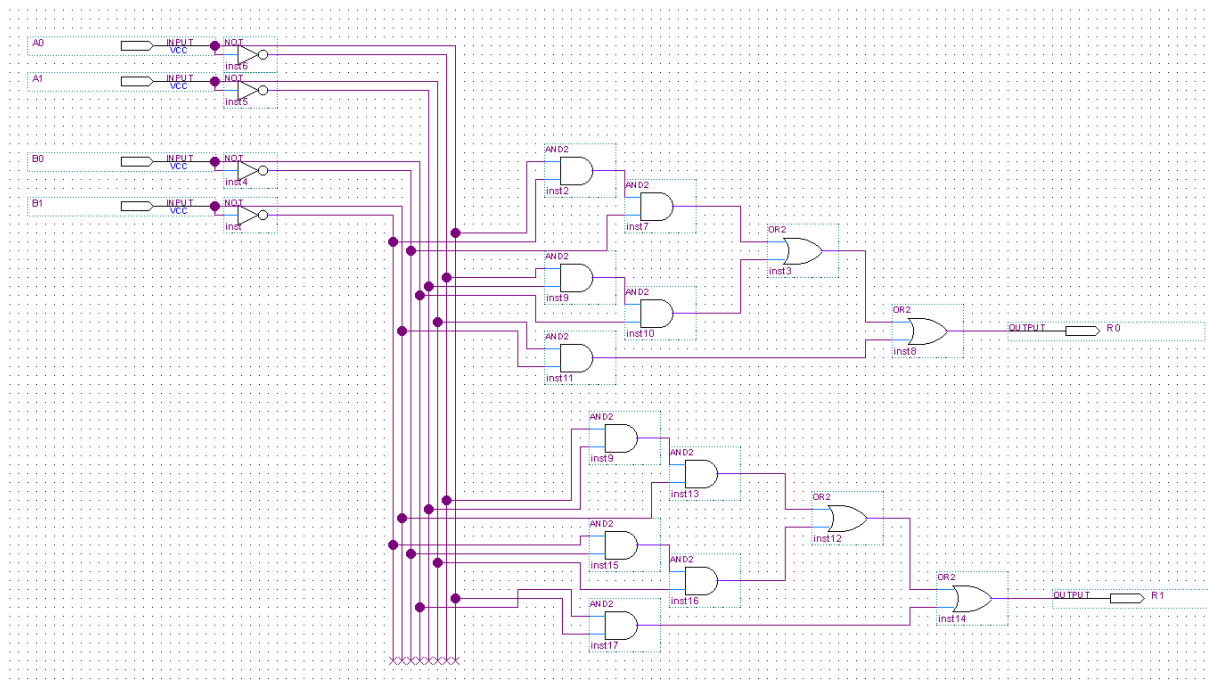
جدول ۴: نقشه ی کارنوی R_1 — سطرها $A_0 A_1$ و ستون ها $B_0 B_1$

R_1	۰۰	۰۱	۱۱	۱۰
۰۰	۰	۰	×	۱
۰۱	۰	۱	×	۰
۱۱	×	×	×	×
۱۰	۱	۰	×	۰

نقشه ی کارنو برای بیت دوم هم فرمول زیر را می دهد:

$$(۴) \quad R_1 = \overline{A_1 A_0} B_1 + A_1 \overline{B_1 B_0} + A_0 B_0$$

پس در نهایت شکل مدار داخلی پیمانه FAX2_MOD_3 به صورت شکل ۲ خواهد بود.



شکل ۲: طرح‌واره‌ی پیمانه FAx2_MOD_3

۲-۵ طراحی پیمانه MOD_3

حالا به طراحی این پیمانه می‌پردازیم، مجدد نقشه‌ی کارنو می‌کشیم و تلاش می‌کنیم ساده‌ترین ترکیب از ورودی‌ها را برای ساخت هر بیت خروجی پیدا کنیم. باید توجه کنیم که برای مقادیر ورودی 10 تا 15 ، چون فرم ورودی به صورت BCD است و این مقادیر در ورودی قرارداد نشده‌اند، پس خروجی بی‌اهمیت است.

جدول ۵: جدول درستی پیمانه MOD_3

رقم	A_3	A_2	A_1	A_0	باقی‌مانده‌ی A بر ۳	$R_1 R_0$
۰	۰	۰	۰	۰	۰	۰ ۰
۱	۰	۰	۱	۰	۱	۰ ۱
۲	۰	۱	۰	۰	۲	۱ ۰
۳	۰	۱	۱	۰	۰	۰ ۰
۴	۱	۰	۰	۰	۱	۰ ۱
۵	۱	۰	۱	۰	۲	۱ ۰
۶	۱	۱	۰	۰	۰	۰ ۰
۷	۱	۱	۱	۰	۱	۰ ۱
۸	۱	۰	۰	۱	۲	۱ ۰
۹	۱	۰	۱	۱	۰	۰ ۰
10 تا 15	۱	—	—	—	—	× ×

جدول ۶: نقشه‌ی کارنوی R_0 — سطرها A_2 A_3 و ستون‌ها A_0 A_1

R_0	۰۰	۰۱	۱۱	۱۰
۰۰	۰	۱	۰	۰
۰۱	۱	۰	۱	۰
۱۱	×	×	×	×
۱۰	۰	۰	×	×

(۵)

$$R_0 = \overline{A_3} \overline{A_2} \overline{A_1} A_0 + \overline{A_2} \overline{A_1} \overline{A_0} + A_2 A_1 A_0$$

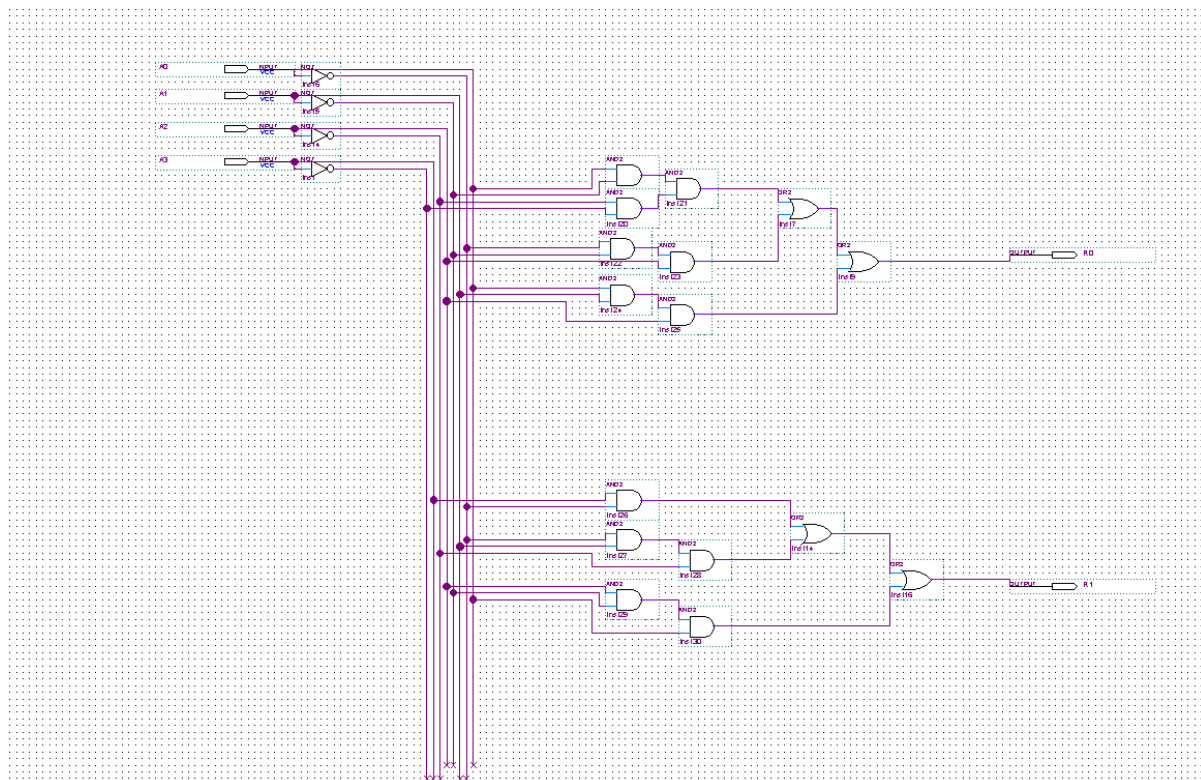
جدول ۷: نقشه‌ی کارنوی R_1 — سطرها A_2 A_3 و ستون‌ها A_0 A_1

R_1	۰۰	۰۱	۱۱	۱۰
۰۰	۰	۰	۰	۱
۰۱	۰	۱	۰	۰
۱۱	×	×	×	×
۱۰	۱	۰	×	×

(۶)

$$R_1 = A_3 \overline{A_0} + \overline{A_2} A_1 \overline{A_0} + A_2 \overline{A_1} A_0$$

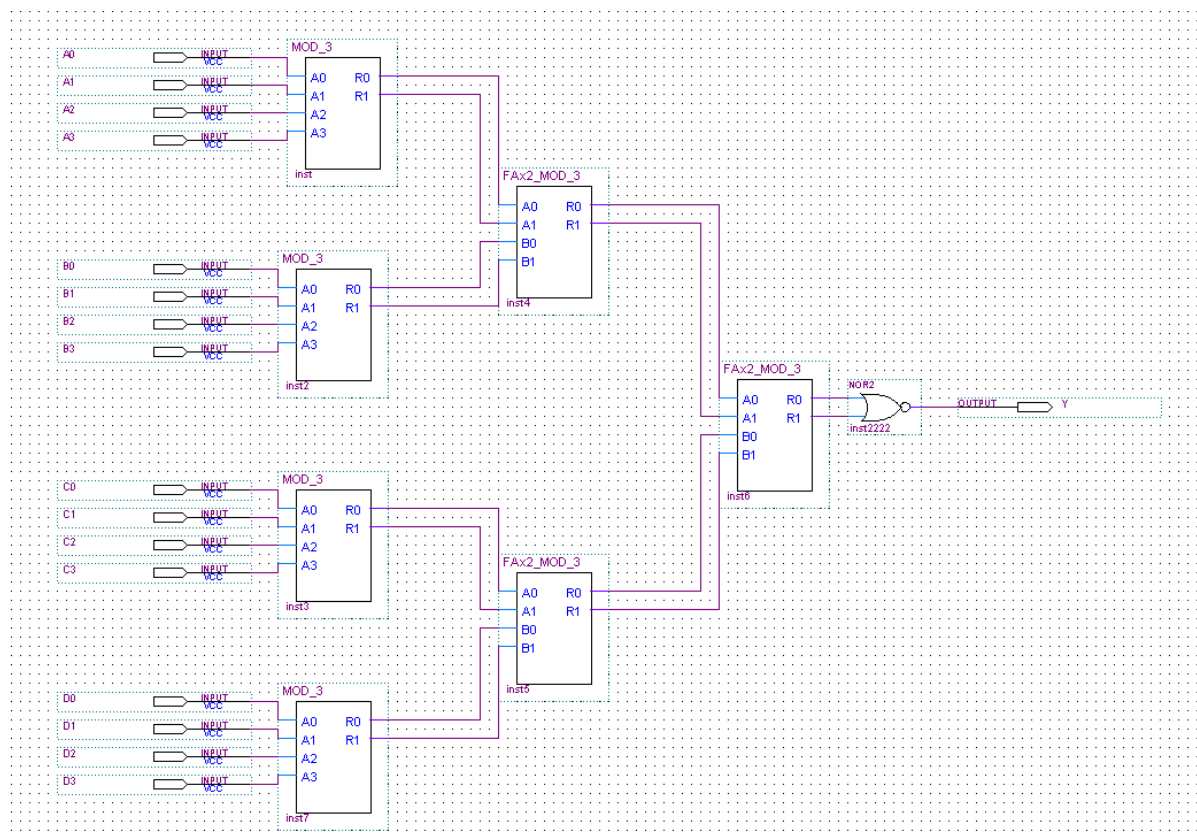
پس در نهایت شکل کلی پیمانه MOD_3 به صورت شکل ۳ می‌شود که دقیقاً از روی نقشه‌ی کارنو ساخته شده است.



شکل ۳: طرح‌واره‌ی پیمانه MOD_3

۲-۶ مدار اصلی

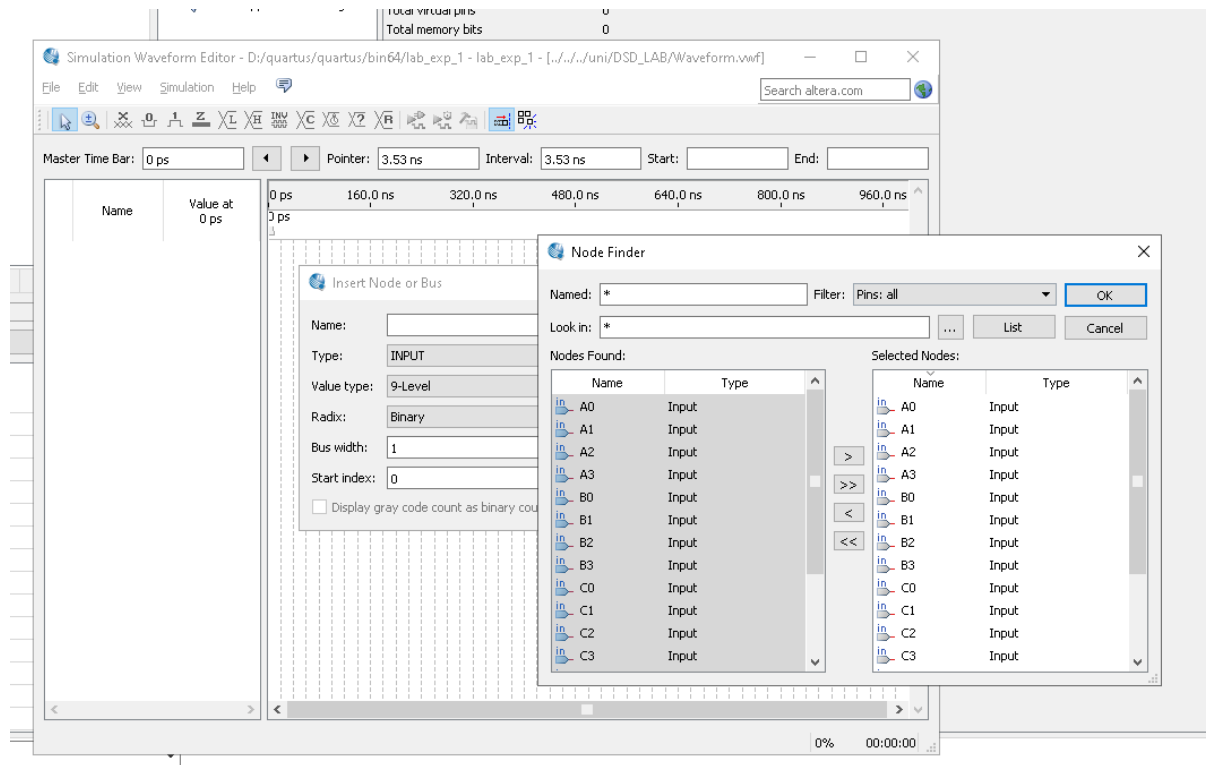
حالا باید مدار اصلی را داخل main.bdf رسم کنیم (طبق همان نمودار درختی)، که در نهایت به صورت شکل ۴ می شود.



شکل ۴: طرح‌واره‌ی مدار اصلی تشخیص بخش‌پذیری بر ۳

برای اینکه بتوانیم سنتز^{۱۰} کنیم و مدار را شبیه‌سازی کنیم، باید ابتدا فایل اصلی را از داخل رابط کاربری Set as top level entity بکنیم، سپس با استفاده از Ctrl + K ابتدا سنتز کنیم و بعد با استفاده از یک فایل شکل موج^{۱۱} با پسوند .vwf تمام گره^{۱۲}‌های مورد نیاز را پیدا کنیم و به داخل Node List اضافه کنیم و در نهایت به آن‌ها مقدار دهیم و به صورت مدار ترکیبی شبیه‌سازی کنیم.

^{۱۰} Synthesis
^{۱۱} Waveform
^{۱۲} Node



شکل ۵: اضافه کردن تمام پین‌ها به شکل موج

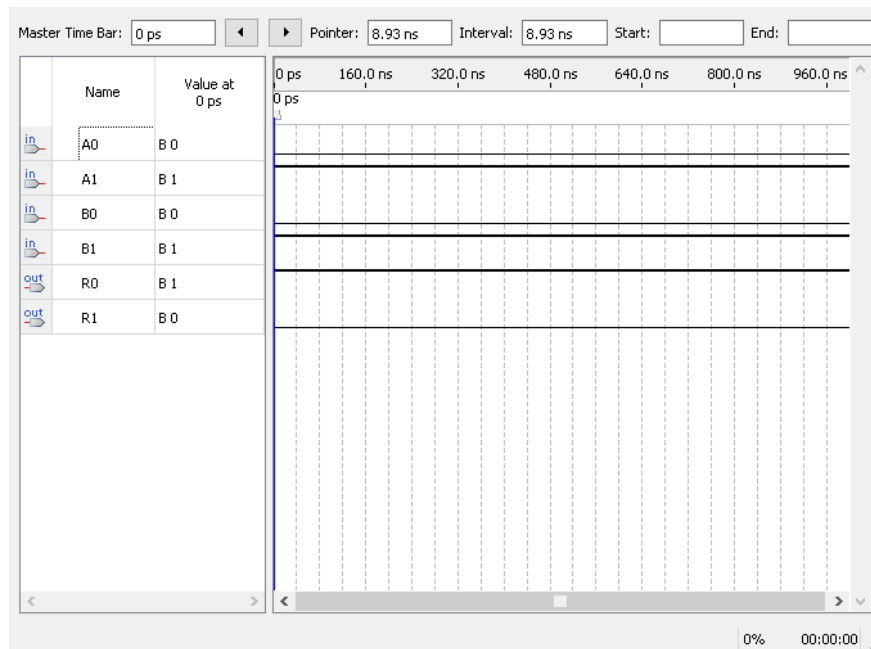
۲-۷ نتایج تست‌ها

برای اینکه از صحت عملکرد مدار اطمینان حاصل کنیم، مدار را مرحله به مرحله تست می‌کنیم و هر بخشی از آن را به صورت مجزا بررسی می‌کنیم.

۲-۷-۱ تست پیمانۀ Fax2_MOD_3

۲-۷-۱-۱ تست ۲ + ۲

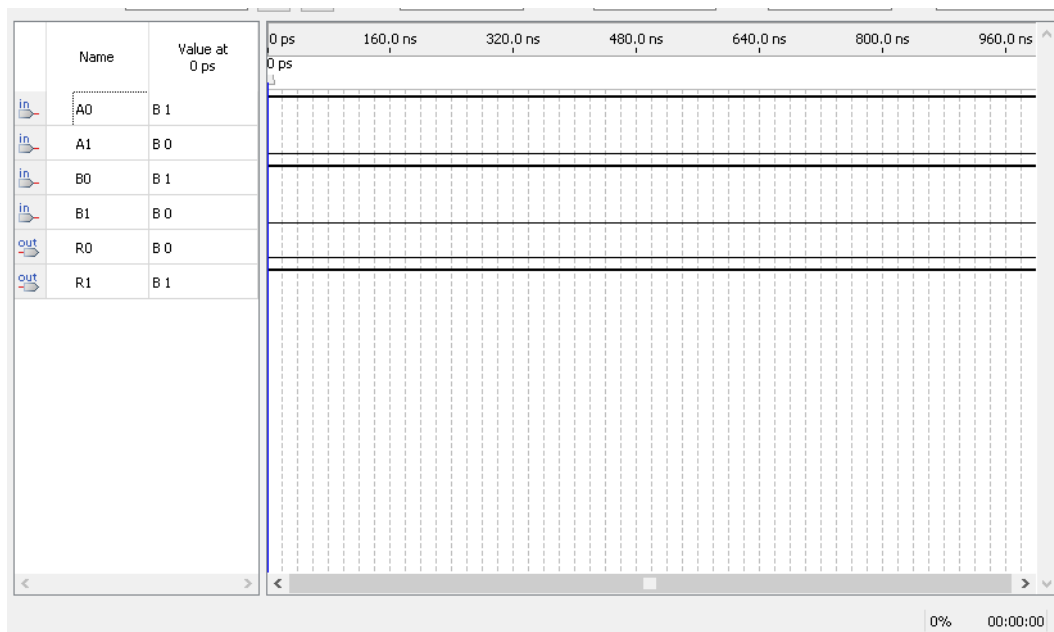
ورودی: $A = ۱۰ (۲)$ ، $B = ۱۰ (۲)$
 محاسبه: $(۲ + ۲) \bmod ۳ = ۱$
 خروجی مورد انتظار: $R = ۰۱ (۱)$



شکل ۶: شبیه‌سازی Fx2_MOD_3 برای ۲ + ۲

۱-۲-۷-۲ تست ۱ + ۱

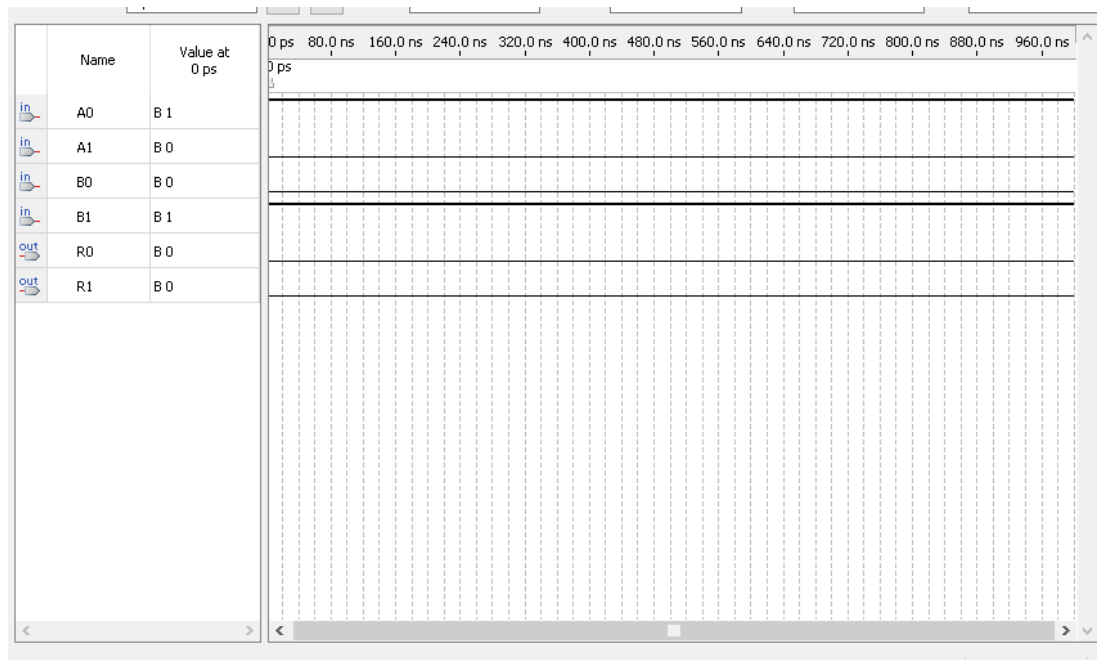
ورودی: $A = 0.1 (1)$, $B = 0.1 (1)$
محاسبه: $(1 + 1) \bmod 3 = 2$
خروجی مورد انتظار: $R = 1.0 (2)$



شکل ۷: شبیه‌سازی Fx2_MOD_3 برای ۱ + ۱

۳-۱-۷-۲ تست ۱ + ۲

ورودی: $A = 01$ (۱)، $B = 10$ (۲)
محاسبه: $(1 + 2) \bmod 3 = 0$
خروجی مورد انتظار: $R = 00$ (۰)

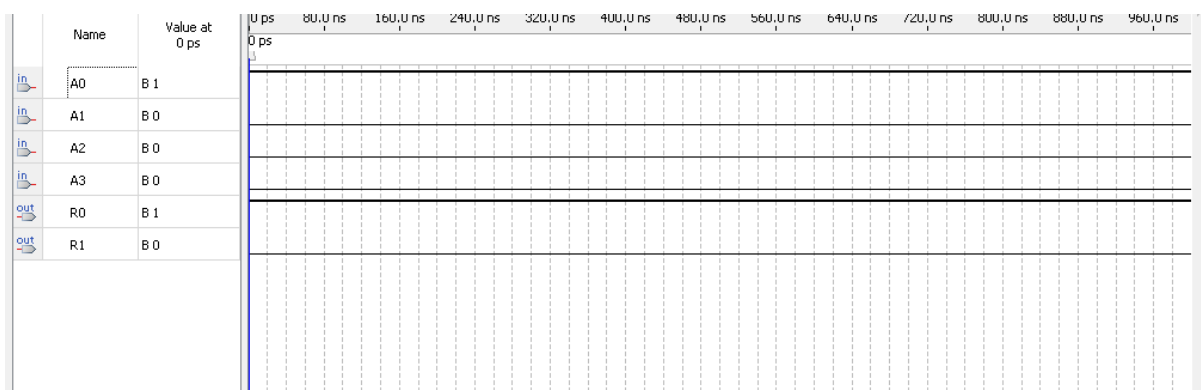


شکل ۸: شبیه‌سازی Fax2_MOD_3 برای ۱ + ۲

۲-۷-۲ تست پیمانه MOD_3

۱-۲-۷-۲ تست ۱

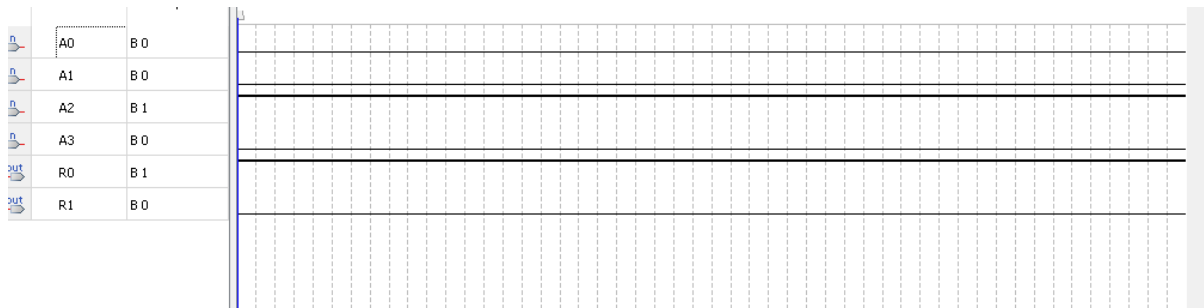
ورودی: $A = 0001$ (۱)
محاسبه: $1 \bmod 3 = 1$
خروجی مورد انتظار: $R = 01$ (۱)



شکل ۹: شبیه‌سازی MOD_3 برای رقم ۱

۲-۷-۲-۲ تست ۴

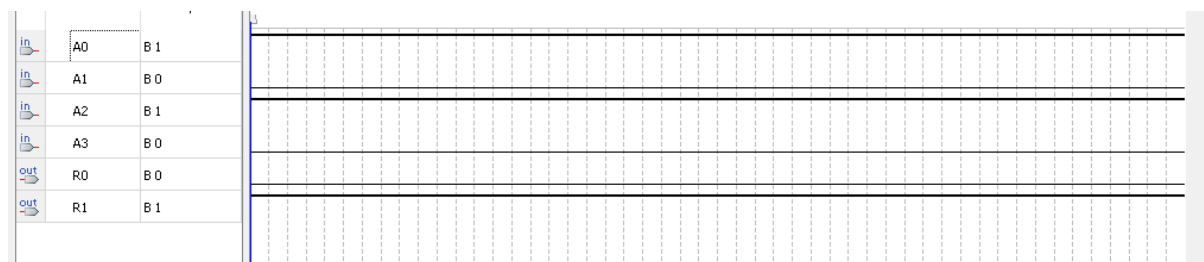
ورودی: (۴) $A = 0100$
 محاسبه: $4 \bmod 3 = 1$
 خروجی مورد انتظار: (۱) $R = 01$



شکل ۱۰: شبیه‌سازی MOD_3 برای رقم ۴

۲-۷-۲-۳ تست ۵

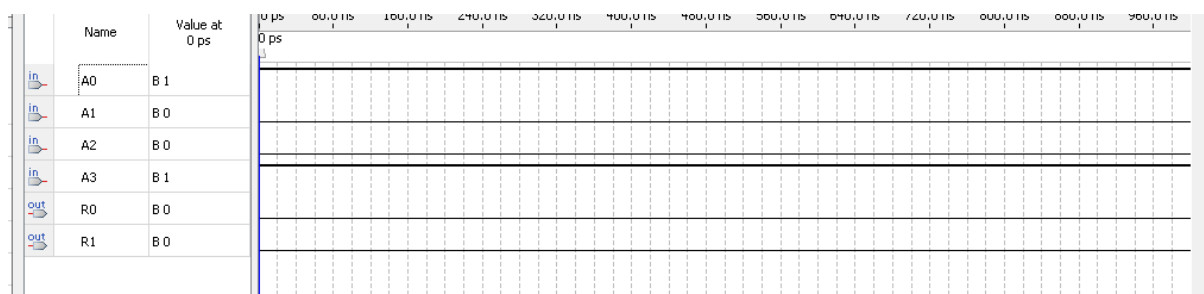
ورودی: (۵) $A = 0101$
 محاسبه: $5 \bmod 3 = 2$
 خروجی مورد انتظار: (۲) $R = 10$



شکل ۱۱: شبیه‌سازی MOD_3 برای رقم ۵

۲-۷-۲-۴ تست ۹

ورودی: (۹) $A = 1001$
 محاسبه: $9 \bmod 3 = 0$
 خروجی مورد انتظار: (۰) $R = 00$



شکل ۱۲: شبیه‌سازی MOD_3 برای رقم ۹

۳-۷-۲ تست مدار کلی

۱-۳-۷-۲ تست ۰۰۰۰

ورودی: $A = 0$ ، $B = 0$ ، $C = 0$ ، $D = 0$ (عدد ۰۰۰۰)
محاسبه: جمع ارقام برابر ۰ است که بر ۳ بخش پذیر است
خروجی مورد انتظار: $Y = 1$

Name	Value at 0 ps	0 ps	40.0 ns	80.0 ns	120.0 ns	160.0 ns	200.0 ns	240.0 ns	280.0 ns	320.0 ns	360.0 ns	400.0 ns	440.0 ns	480.0 ns	520.0 ns	560.0 ns	600.0 ns	640.0 ns	680.0 ns	720.0 ns	760.0 ns	800.0 ns	840.0 ns	880.0 ns	920.0 ns	960.0 ns	1.0 us
A0	0 0																										
A1	0 0																										
A2	0 0																										
A3	0 0																										
B0	0 0																										
B1	0 0																										
B2	0 0																										
B3	0 0																										
C0	0 0																										
C1	0 0																										
C2	0 0																										
C3	0 0																										
D0	0 0																										
D1	0 0																										
D2	0 0																										
D3	0 0																										
Y	0 1																										

شکل ۱۳: شبیه سازی مدار بخش پذیری بر ۳ برای عدد ۰۰۰۰

۲-۳-۷-۲ تست ۰۰۹۹

ورودی: $A = 9$ ، $B = 9$ ، $C = 0$ ، $D = 0$ (عدد ۰۰۹۹)
محاسبه: جمع ارقام برابر ۱۸ است که بر ۳ بخش پذیر است
خروجی مورد انتظار: $Y = 1$

Name	Value at 0 ps	0 ps	40.0 ns	80.0 ns	120.0 ns	160.0 ns	200.0 ns	240.0 ns	280.0 ns	320.0 ns	360.0 ns	400.0 ns	440.0 ns	480.0 ns	520.0 ns	560.0 ns	600.0 ns	640.0 ns	680.0 ns	720.0 ns	760.0 ns	800.0 ns	840.0 ns	880.0 ns	920.0 ns	960.0 ns	1.0 us
A0	0 1																										
A1	0 0																										
A2	0 0																										
A3	0 1																										
B0	0 1																										
B1	0 0																										
B2	0 0																										
B3	0 1																										
C0	0 0																										
C1	0 0																										
C2	0 0																										
C3	0 0																										
D0	0 0																										
D1	0 0																										
D2	0 0																										
D3	0 0																										
Y	0 1																										

شکل ۱۴: شبیه سازی مدار بخش پذیری بر ۳ برای عدد ۰۰۹۹

۳-۳-۷-۲ تست ۰۵۷۶

ورودی: $A = 6$ ، $B = 7$ ، $C = 5$ ، $D = 0$ (عدد ۰۵۷۶)
محاسبه: جمع ارقام برابر ۱۸ است که بر ۳ بخش پذیر است
خروجی مورد انتظار: $Y = 1$

Name	Value at 0 ps	0 ps	40.0 ns	80.0 ns	120.0 ns	160.0 ns	200.0 ns	240.0 ns	280.0 ns	320.0 ns	360.0 ns	400.0 ns	440.0 ns	480.0 ns	520.0 ns	560.0 ns	600.0 ns	640.0 ns	680.0 ns	720.0 ns	760.0 ns	800.0 ns	840.0 ns	880.0 ns	920.0 ns	960.0 ns	1.0 us
A0	0 0																										
A1	0 1																										
A2	0 1																										
A3	0 0																										
B0	0 1																										
B1	0 1																										
B2	0 1																										
B3	0 0																										
C0	0 1																										
C1	0 0																										
C2	0 1																										
C3	0 0																										
D0	0 0																										
D1	0 0																										
D2	0 0																										
D3	0 0																										
Y	0 1																										

شکل ۱۵: شبیه‌سازی مدار بخش‌پذیری بر ۳ برای عدد ۵۷۶°

۳-۷-۲ تست ۳۵۷۶

ورودی: $A = ۶$, $B = ۷$, $C = ۵$, $D = ۳$ (عدد ۳۵۷۶)
محاسبه: جمع ارقام برابر ۲۱ است که بر ۳ بخش‌پذیر است
خروجی مورد انتظار: $Y = ۱$

Name	Value at 0 ps	0 ps	40.0 ns	80.0 ns	120.0 ns	160.0 ns	200.0 ns	240.0 ns	280.0 ns	320.0 ns	360.0 ns	400.0 ns	440.0 ns	480.0 ns	520.0 ns	560.0 ns	600.0 ns	640.0 ns	680.0 ns	720.0 ns	760.0 ns	800.0 ns	840.0 ns	880.0 ns	920.0 ns	960.0 ns	1.0 us
A0	0 0																										
A1	0 1																										
A2	0 1																										
A3	0 0																										
B0	0 1																										
B1	0 1																										
B2	0 1																										
B3	0 0																										
C0	0 1																										
C1	0 0																										
C2	0 1																										
C3	0 0																										
D0	0 1																										
D1	0 1																										
D2	0 0																										
D3	0 0																										
Y	0 1																										

شکل ۱۶: شبیه‌سازی مدار بخش‌پذیری بر ۳ برای عدد ۳۵۷۶

۳-۷-۲ تست ۹۵۷۶

ورودی: $A = ۶$, $B = ۷$, $C = ۵$, $D = ۹$ (عدد ۹۵۷۶)
محاسبه: جمع ارقام برابر ۲۷ است که بر ۳ بخش‌پذیر است
خروجی مورد انتظار: $Y = ۱$

Name	Value at 0 ps	0 ps	40.0 ns	80.0 ns	120.0 ns	160.0 ns	200.0 ns	240.0 ns	280.0 ns	320.0 ns	360.0 ns	400.0 ns	440.0 ns	480.0 ns	520.0 ns	560.0 ns	600.0 ns	640.0 ns	680.0 ns	720.0 ns	760.0 ns	800.0 ns	840.0 ns	880.0 ns	920.0 ns	960.0 ns	1.0 us
A0	0 0																										
A1	0 1																										
A2	0 1																										
A3	0 0																										
B0	0 1																										
B1	0 1																										
B2	0 1																										
B3	0 0																										
C0	0 1																										
C1	0 0																										
C2	0 1																										
C3	0 0																										
D0	0 1																										
D1	0 0																										
D2	0 0																										
D3	0 1																										
Y	0 1																										

شکل ۱۷: شبیه‌سازی مدار بخش‌پذیری بر ۳ برای عدد ۹۵۷۶

ورودی: $A = 1, B = 1, C = 0, D = 8$ (عدد ۸۰۱۱)
محاسبه: جمع ارقام برابر 10 است که بر 3 بخش پذیر نیست
خروجی مورد انتظار: $Y = 0$

Name	Value at 0 ps	0 ps	40,0 ns	80,0 ns	120,0 ns	160,0 ns	200,0 ns	240,0 ns	280,0 ns	320,0 ns	360,0 ns	400,0 ns	440,0 ns	480,0 ns	520,0 ns	560,0 ns	600,0 ns	640,0 ns
in A0	B 1																	
in A1	B 0																	
in A2	B 0																	
in A3	B 0																	
in B0	B 1																	
in B1	B 0																	
in B2	B 0																	
in B3	B 0																	
in C0	B 0																	
in C1	B 0																	
in C2	B 0																	
in C3	B 0																	
in D0	B 0																	
in D1	B 0																	
in D2	B 0																	
in D3	B 1																	
out Y	B 0																	

شکل ۱۸: شبیه سازی مدار بخش پذیری بر ۳ برای عدد ۸۰۱۱

۳ بخش دوم: تشخیص بخش پذیری بر ۱۱

۳-۱ ایده کلی و اثبات ریاضی

همان فرمول های بخش اول ولی برای باقی مانده ی ۱۱. این بار 10 در پیمانه ۱۱ باقی مانده ۱- دارد، پس توان های 10^k یک در میان علامت عوض می کنند.

لم ۲. برای عدد N با ارقام دهدهی $DCBA$ داریم $N \equiv A - B + C - D \pmod{11}$.

اثبات. چون $10 \equiv -1 \pmod{11}$ ، برای هر $k \geq 0$ داریم $10^k \equiv (-1)^k \pmod{11}$. پس

$$(7) \quad N = D \times 10^3 + C \times 10^2 + B \times 10^1 + A \equiv A - B + C - D \pmod{11}$$

□

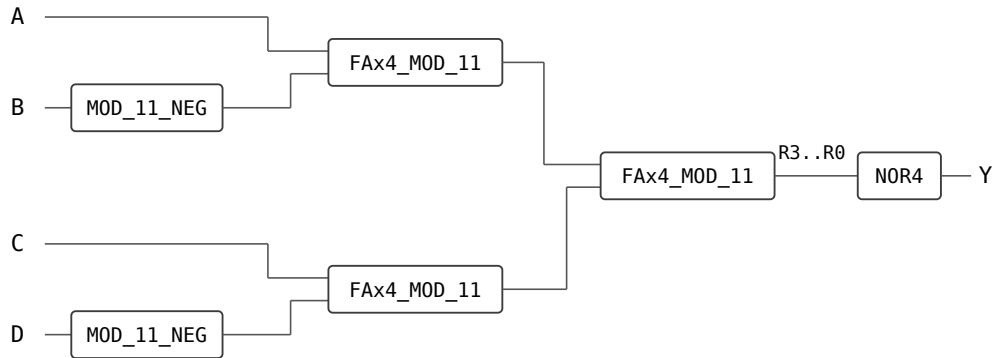
برای اینکه با اعداد منفی سروکار نداشته باشیم، تفریق را به جمع قرینه در پیمانه ۱۱ تبدیل می کنیم:

$$(8) \quad N \equiv A + (11 - B) + C + (11 - D) \pmod{11}$$

این سوال هم تقریباً با ایده ی بخش قبلی حل می شود، ولی در بخش اول رقم اول و سوم تحت تاثیر باقی مانده گیری خودشان می شوند و رقم دوم و چهارم عملاً باقی مانده بر ۱۱ منفی آن ها اهمیت دارد، پس باید برای این یک پیمانه تعریف کنیم. نکته این است که در تبدیل به باقی مانده ۱۱، از فرم استاندارد BCD خارج شدن اهمیتی ندارد و چون می شود تا عدد ۱۵ را در ۴ بیت نمایش داد از این مورد استفاده می کنیم؛ پس ایرادی ندارد که در حین کار از فرم BCD خارج شویم، چون اصلاً برای پردازش ما اهمیتی نباید داشته باشد. سپس برای تلفیق آن ها مجدد باید یک فرایند درختی انجام بدهیم و در هر مرحله جمع ۲ عدد ۴ بیتی را بگیریم و خروجی باقی مانده بر ۱۱ مجموع آن ها را خروجی بدهیم.

همچنین حواسمان هست که در هر مرحله‌ی داخلی هر عددی که ما داریم باقی‌مانده بر ۱۱ است، پس حداکثر مقدار ۱۰ را می‌تواند اخذ کند، که یعنی جمع دو عدد در بین مدارها حداکثر ۲۰ است که در ۵ بیت جا می‌شود.

۳-۲ ساختار کلی مدار



شکل ۱۹: ساختار درختی مدار تشخیص بخش‌پذیری بر ۱۱

۱. مرحله اول: ارقام دهگان و هزارگان (که ضریبشان در بسط عدد منفی است) وارد پیمانه MOD_11_NEG می‌شوند و به قرینه‌ی خودش در پیمانه ۱۱ تبدیل می‌شوند. ارقام یکان و صدگان بدون تغییر عبور می‌کنند، چون خودش از ۱۱ کوچک‌تر هستند و از قبل باقی‌مانده بر ۱۱ خودش هستند.
۲. مرحله دوم: نتیجه‌ها دوبه‌دو وارد پیمانه Fx4_MOD_11 می‌شوند.
۳. مرحله سوم: حاصل این دو وارد آخرین Fx4_MOD_11 می‌شود تا باقی‌مانده کل عدد بر ۱۱ محاسبه شود.
۴. مرحله نهایی: اگر خروجی نهایی برابر ۰۰۰۰ باشد یعنی باقی‌مانده صفر است، پس با یک دروازه‌ی NOR چهار ورودی روی R3 و R0 خروجی Y را می‌سازیم.

۳-۳ طراحی پیمانه MOD_11_NEG

این پیمانه یک رقم BCD می‌گیرد و قرینه‌ی آن به پیمانه ۱۱ را برمی‌گرداند. جدول درستی آن در جدول ۸ آمده است. مجدد مثل بخش اول سوال باید حواسمان به مقادیر بی‌اهمیت باشد؛ ورودی این پیمانه یک رقم BCD است پس ۱۰ تا ۱۵ اصلاً اتفاق نمی‌افتند.

جدول ۸: جدول درستی پیمانه MOD_11_NEG

رقم	A_3	A_2	A_1	A_0	قرینه‌ی A به پیمانه ۱۱	$R_3R_2R_1R_0$
۰	۰	۰	۰	۰	۰	۰۰۰۰
۱	۰	۰	۰	۱	۱۰	۱۰۱۰
۲	۰	۰	۱	۰	۹	۱۰۰۱
۳	۰	۰	۱	۱	۸	۱۰۰۰
۴	۰	۱	۰	۰	۷	۰۱۱۱
۵	۰	۱	۰	۱	۶	۰۱۱۰
۶	۰	۱	۱	۰	۵	۰۱۰۱
۷	۰	۱	۱	۱	۴	۰۱۰۰
۸	۱	۰	۰	۰	۳	۰۰۱۱
۹	۱	۰	۰	۱	۲	۰۰۱۰
۱۰ تا ۱۵	۱	—	—	—	—	× × × ×

جدول ۹: نقشه‌ی کارنوی R_0 — سطرها A_2 A_3 و ستون‌ها A_0 A_1

R_0	۰۰	۰۱	۱۱	۱۰
۰۰	۰	۰	۰	۱
۰۱	۱	۰	۰	۱
۱۱	×	×	×	×
۱۰	۱	۰	×	×

$$(۹) \quad R_0 = A_1 \overline{A_0} + A_2 \overline{A_0} + A_3 \overline{A_0} = \overline{A_0} (A_1 + A_2 + A_3)$$

جدول ۱۰: نقشه‌ی کارنوی R_1 — سطرها A_2 A_3 و ستون‌ها A_0 A_1

R_1	۰۰	۰۱	۱۱	۱۰
۰۰	۰	۱	۰	۰
۰۱	۱	۱	۰	۰
۱۱	×	×	×	×
۱۰	۱	۱	×	×

$$(۱۰) \quad R_1 = \overline{A_1} A_0 + A_2 \overline{A_1} + A_3 \overline{A_1} = \overline{A_1} (A_0 + A_2 + A_3)$$

جدول ۱۱: نقشه‌ی کارنوی R_2 — سطرها A_2 A_3 و ستون‌ها A_0 A_1

R_2	۰۰	۰۱	۱۱	۱۰
۰۰	۰	۰	۰	۰
۰۱	۱	۱	۱	۱
۱۱	×	×	×	×
۱۰	۰	۰	×	×

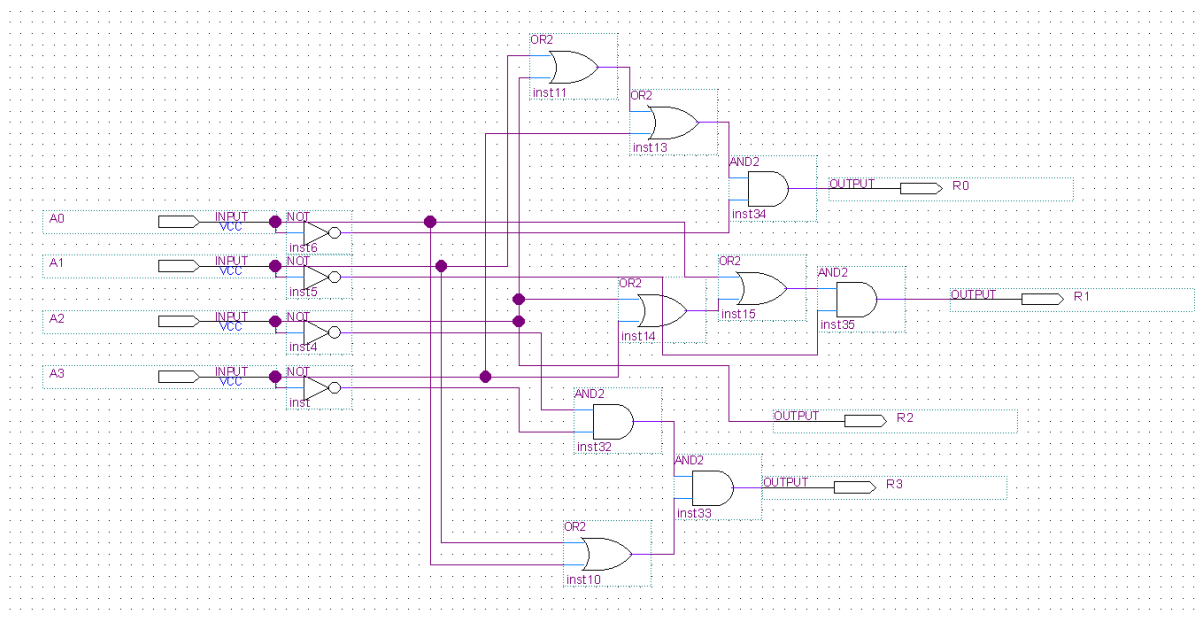
$$(۱۱) \quad R_2 = A_2$$

جدول ۱۲: نقشه‌ی کارنوی R_3 — سطرها A_2 A_3 و ستون‌ها A_0 A_1

R_3	۰۰	۰۱	۱۱	۱۰
۰۰	۰	۱	۱	۱
۰۱	۰	۰	۰	۰
۱۱	×	×	×	×
۱۰	۰	۰	×	×

$$(۱۲) \quad R_3 = \overline{A_3} \overline{A_2} A_1 + \overline{A_3} \overline{A_2} A_0 = \overline{A_3} \overline{A_2} (A_1 + A_0)$$

طبق نقشه‌های کارنو، مدار به صورت شکل ۲۰ خواهد بود.



شکل ۲۰: طرح‌واره‌ی پیمانه MOD_11_NEG

۳-۴ طراحی پیمانه $Fax4_MOD_11$

نحوه‌ی عملکرد این مدار خیلی شبیه عملکرد مدار جمع‌کننده‌ی دو عدد BCD است (ولی مدار هیچ ربطی به نمایش BCD ندارد و نباید گیج شد). به این صورت عمل می‌کند که ابتدا دو عدد را به صورت عادی (۴ بیتی) جمع می‌زنیم و

در مرحله‌ی بعد اگر مشکلی پیش آمد اصلاحیه انجام می‌دهیم. کل هدف این است که باقی‌مانده بر ۱۱ بگیریم، پس بعد از جمع زدن در صورتی که جمع از ۱۱ بیشتر شد می‌توانیم جواب نهایی را به نحوی اصلاح کنیم که انگار باقی‌مانده بر ۱۱ گرفتیم؛ عملاً باید از جواب نهایی ۱۱ تا کم کنیم، که به پیمانه ۱۶ (یعنی همان ۴ بیت) معادل است با اضافه کردن ۵ به جواب. اگر بیت نقلی^{۱۳} هم داشتیم می‌شود به سادگی آن را در نظر نگرفت، زیرا که در باقی‌مانده بر ۱۱ اهمیت نخواهد داشت.

مرحله به مرحله:

۱. اول با یک FAX4 (جمع‌کننده‌ی کامل ۴ بیتی) مجموع $S = A + B$ را می‌گیریم. چون هر دو ورودی حداکثر ۱۰ هستند، S حداکثر ۲۰ می‌شود که با C_OUT و S3 نمایش داده می‌شود.

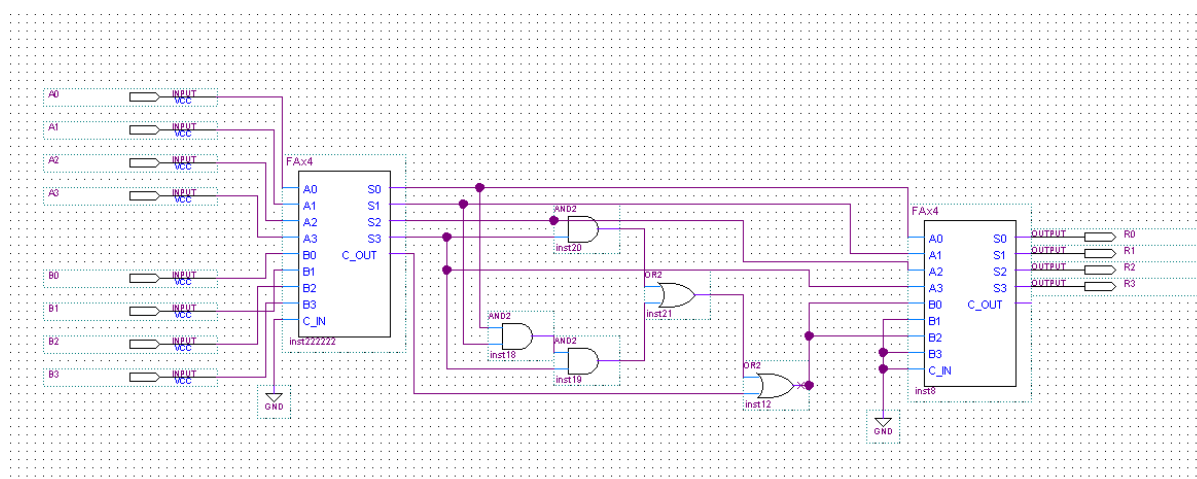
۲. بعد باید تشخیص بدهیم که آیا $S \geq 11$ است یا نه. اگر C_OUT یک باشد که $S \geq 16$ است و حتماً نیاز به اصلاح داریم. در غیر این صورت باید ببینیم آیا S3.S0 یکی از مقادیر ۱۱ تا ۱۵ است یا نه، که این حالت‌ها دقیقاً همان‌هایی هستند که S_3 روشن است و همزمان S_2 یا $(S_1$ و $S_0)$ روشن باشند. پس سیگنال اصلاح به این شکل در می‌آید:

$$(13) \quad C = C_{out} + S_3 S_2 + S_3 S_1 S_0$$

۳. در آخر یک FAX4 دیگر می‌گذاریم که اگر C برقرار بود عدد ۵ (یعنی ۰۱۰۱) را به S اضافه کند، پس C را مستقیم به ورودی‌های B0 و B2 این جمع‌کننده وصل می‌کنیم و بقیه‌ی ورودی‌ها را صفر می‌گذاریم. بیت نقلی این مرحله هم دور ریخته می‌شود، چون جمع در پیمانه ۱۶ دقیقاً همان کم کردن ۱۱ را به ما می‌دهد:

$$(14) \quad R = (S + 5C) \bmod 16$$

چون $S \leq 20$ است، بعد از اصلاح حداکثر مقدار خروجی $9 = 11 - 20$ می‌شود و همیشه در بازه‌ی ۰ تا ۱۰ باقی می‌ماند، پس خروجی این پیمانه دوباره یک باقی‌مانده معتبر بر ۱۱ است و می‌شود آن را به مرحله‌ی بعدی درخت داد.

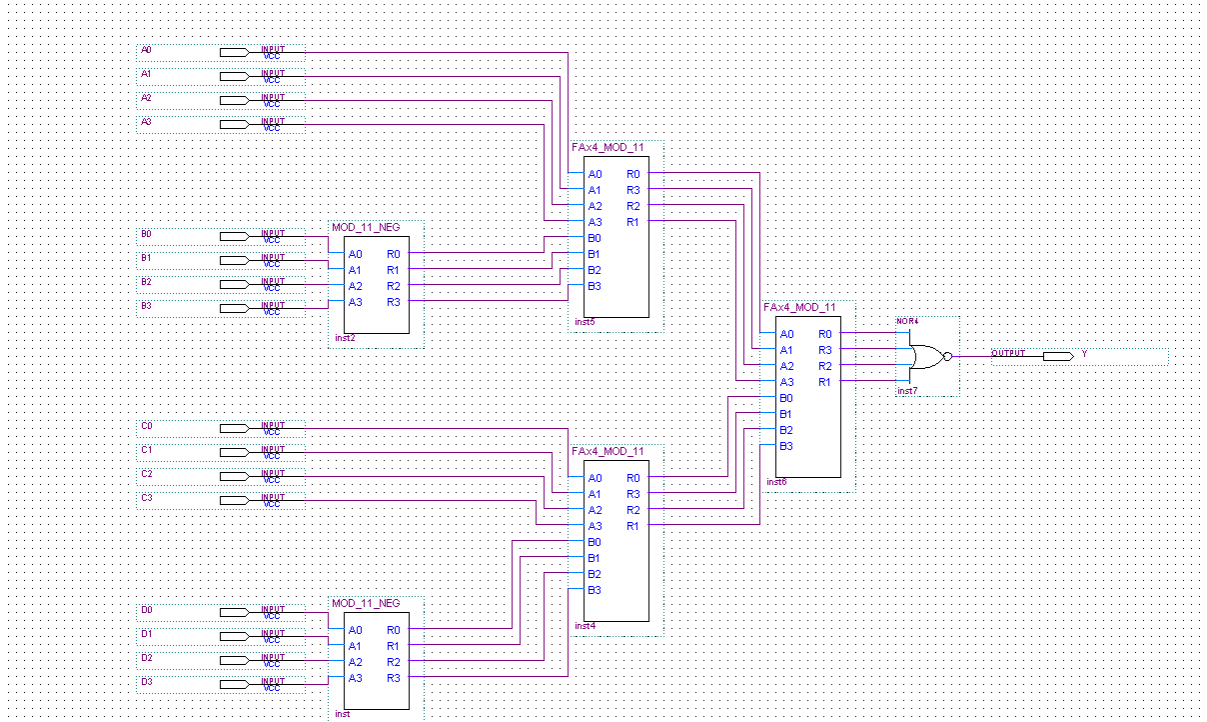


شکل ۲۱: طرح‌واره‌ی پیمانه FAX4_MOD_11

^{۱۳} Carry

۳-۵ مدار اصلی

مدار اصلی نیز مطابق نمودار درختی رسم می‌شود و به صورت شکل ۲۲ خواهد بود؛ فقط برای دروازه‌ی آخر از دروازه‌ی کاملاً پایه و بدیهی استفاده نشده و از یک NOR چهار ورودی استفاده شده است (صرفاً برای تمیزی، وگرنه به سادگی می‌شد با دروازه‌های تک‌ورودی نیز پیاده‌سازی شود).



شکل ۲۲: طرح‌واره‌ی مدار اصلی تشخیص بخش‌پذیری بر ۱۱

۳-۶ نتایج تست‌ها

۳-۶-۱ تست پیمانه MOD_11_NEG

۳-۶-۱-۱ تست ۰

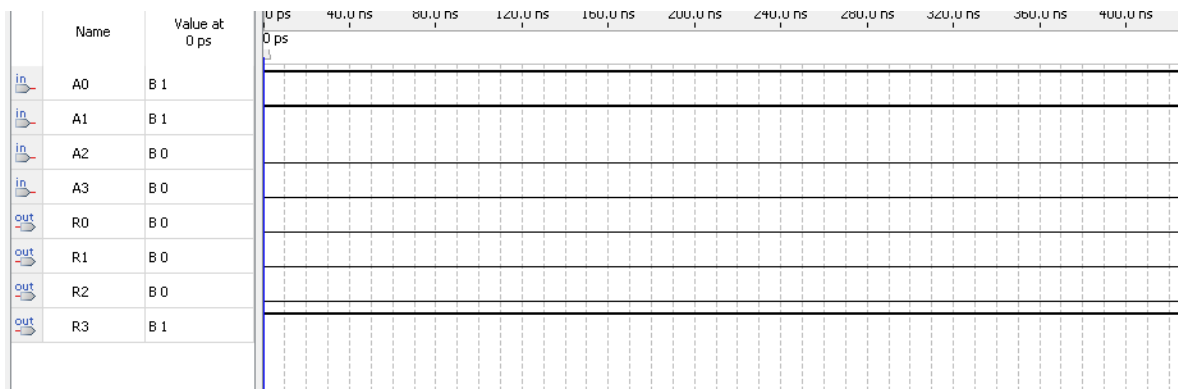
ورودی: $A = \dots (0)$
 محاسبه: $(0 - 0) \bmod 11 = 0$
 خروجی مورد انتظار: $R = \dots (0)$



شکل ۲۳: شبیه‌سازی MOD_11_NEG برای رقم ۰

۳-۶-۱-۲ تست ۳

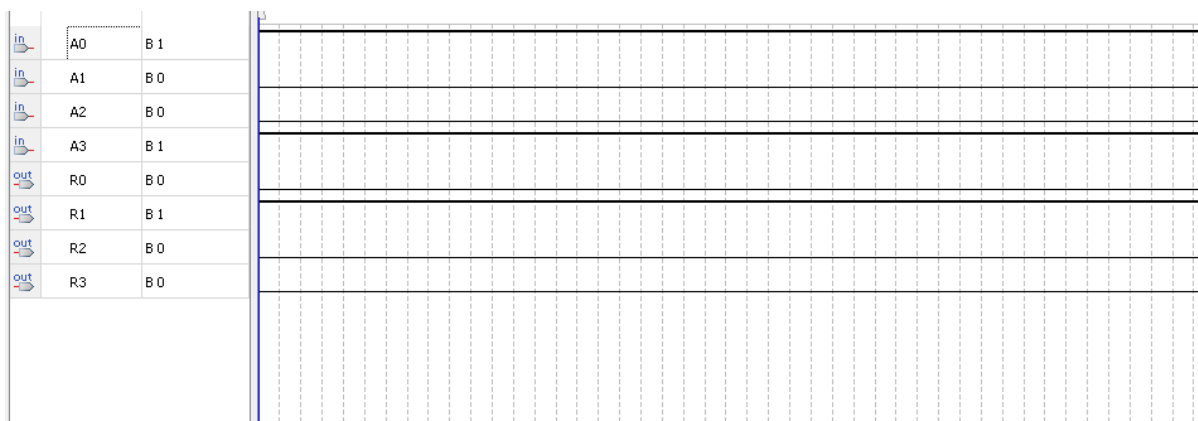
ورودی: $A = 0011$ (۳)
محاسبه: $(11 - 3) \bmod 11 = 8$
خروجی مورد انتظار: $R = 1000$ (۸)



شکل ۲۴: شبیه‌سازی MOD_11_NEG برای رقم ۳

۳-۶-۱-۳ تست ۹

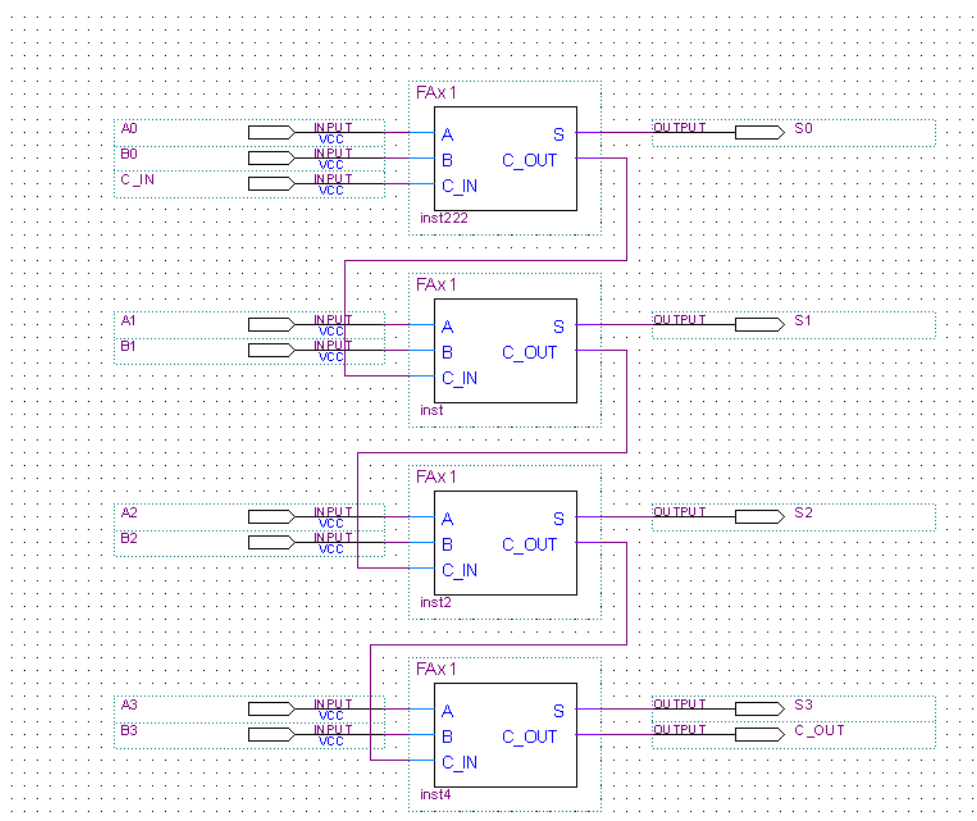
ورودی: $A = 1001$ (۹)
محاسبه: $(11 - 9) \bmod 11 = 2$
خروجی مورد انتظار: $R = 0010$ (۲)



شکل ۲۵: شبیه‌سازی MOD_11_NEG برای رقم ۹

۲-۶-۳ تست پیمانه FAX4

شکل مدار جمع‌کننده‌ی کامل ۴ بیتی در شکل ۲۶ آمده است.



شکل ۲۶: طرح‌واره‌ی جمع‌کننده‌ی کامل ۴ بیتی

۱-۲-۶-۳ تست ۹ + ۹

ورودی: $A = 1001 (9)$, $B = 1001 (9)$, $C_IN = 0$
 محاسبه: $9 + 9 = 18$
 خروجی مورد انتظار: $C_OUT = 1$ و $S = 0010$ (روی هم ۱۸ در پنج بیت)

Name	Value at 0 ps	40,0 ns	80,0 ns	120,0 ns	160,0 ns	200,0 ns	240,0 ns	280,0 ns	320,0 ns	360,0 ns	400,0 ns	440,0 ns	480,0 ns	520,0 ns	560,0 ns	600,0 ns	640,0 ns	680,0 ns	720,0 ns	760,0 ns	800,0 ns	840,0 ns	880,0 ns	920,0 ns	960,0 ns	1,0 us
A0	B 1																									
A1	B 0																									
A2	B 0																									
A3	B 1																									
B0	B 1																									
B1	B 0																									
B2	B 0																									
B3	B 1																									
C_IN	B 0																									
C_OUT	B 1																									
S0	B 0																									
S1	B 1																									
S2	B 0																									
S3	B 0																									

شکل ۲۷: شبیه‌سازی جمع‌کننده‌ی ۴ بیتی برای ۹ + ۹

از نوشتن تست‌های بیشتر خودداری کردیم، زیرا که مدار جمع‌کننده‌ی کامل ۴ بیتی مرسوم و شناخته‌شده است و تست کوچک کفاف می‌دهد.

۳-۶-۳ تست پیمانه FAx4_MOD_11

۱-۳-۶-۳ تست ۵ + ۵

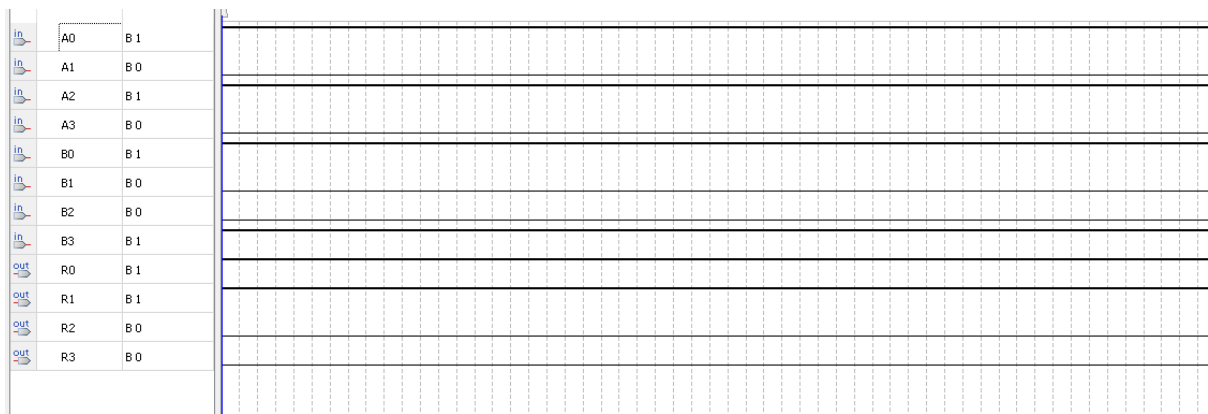
ورودی: $A = 0101 (5)$ ، $B = 0000 (0)$
محاسبه: $(5 + 0) \bmod 11 = 5$
خروجی مورد انتظار: $R = 0101 (5)$

Name	Value at 0 ps	40,0 ns	80,0 ns	120,0 ns	160,0 ns	200,0 ns	240,0 ns	280,0 ns	320,0 ns	360,0 ns	400,0 ns	440,0 ns	480,0 ns	520,0 ns	560,0 ns	600,0 ns	640,0 ns	680,0 ns	720,0 ns	760,0 ns	800,0 ns	840,0 ns	880,0 ns	920,0 ns	960,0 ns	1,0 us
A0	B 1																									
A1	B 0																									
A2	B 1																									
A3	B 0																									
B0	B 0																									
B1	B 0																									
B2	B 0																									
B3	B 0																									
R0	B 1																									
R1	B 0																									
R2	B 1																									
R3	B 0																									

شکل ۲۸: شبیه‌سازی FAx4_MOD_11 برای ۵ + ۵

۲-۳-۶-۳ تست ۵ + ۹

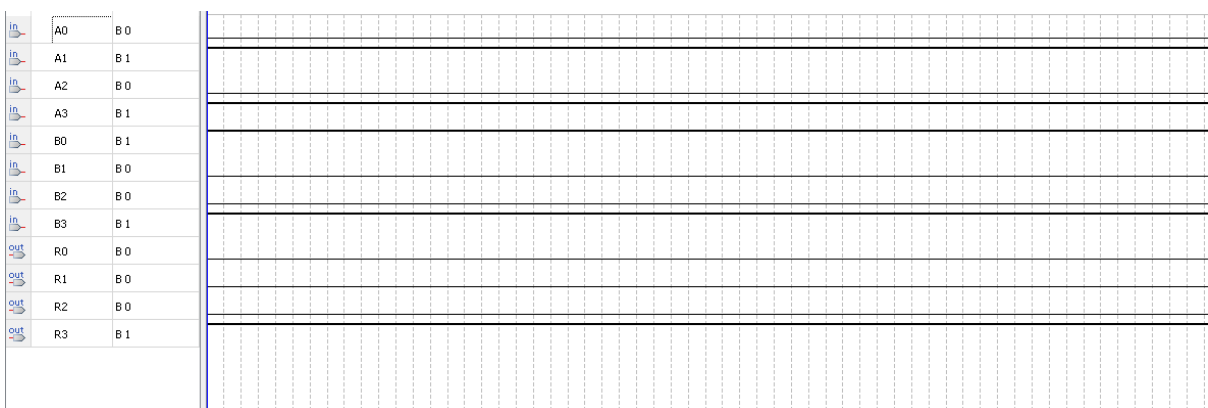
ورودی: $A = 0101 (5)$ ، $B = 1001 (9)$
محاسبه: $(5 + 9) \bmod 11 = 3$
خروجی مورد انتظار: $R = 0011 (3)$



شکل ۲۹: شبیه‌سازی Fax4_MOD_11 برای ۵ + ۹

۳-۳-۶ تست ۹ + ۱۰

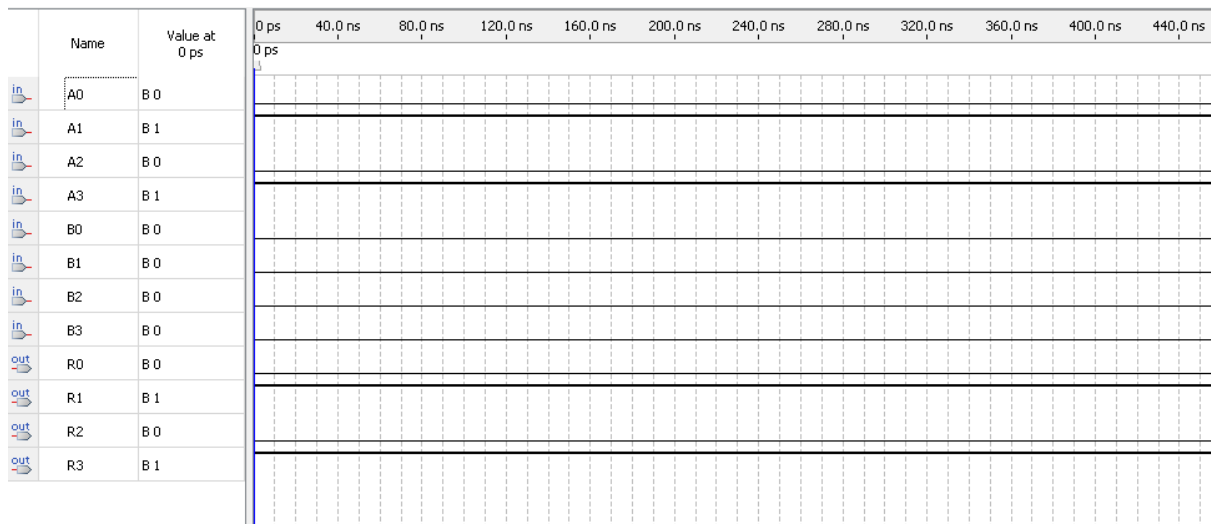
ورودی: $A = 1001 (9)$, $B = 1010 (10)$
محاسبه: $(9 + 10) \bmod 11 = 8$
خروجی مورد انتظار: $R = 1000 (8)$



شکل ۳۰: شبیه‌سازی Fax4_MOD_11 برای ۹ + ۱۰

۳-۳-۶ تست ۱۰ + ۰

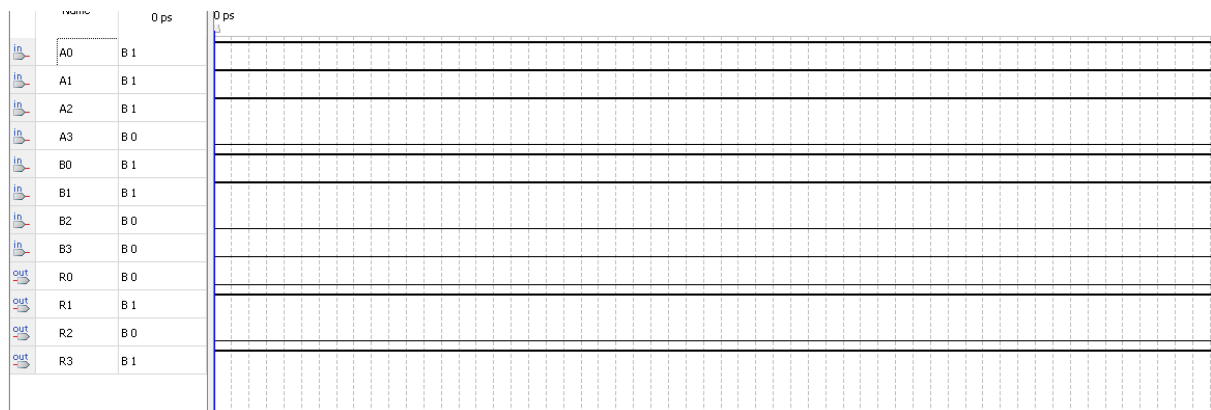
ورودی: $A = 1010 (10)$, $B = 0000 (0)$
محاسبه: $(10 + 0) \bmod 11 = 10$
خروجی مورد انتظار: $R = 1010 (10)$



شکل ۳۱: شبیه‌سازی Fax4_MOD_11 برای $10 + 0$

۳-۶-۳-۵ تست $3 + 7$

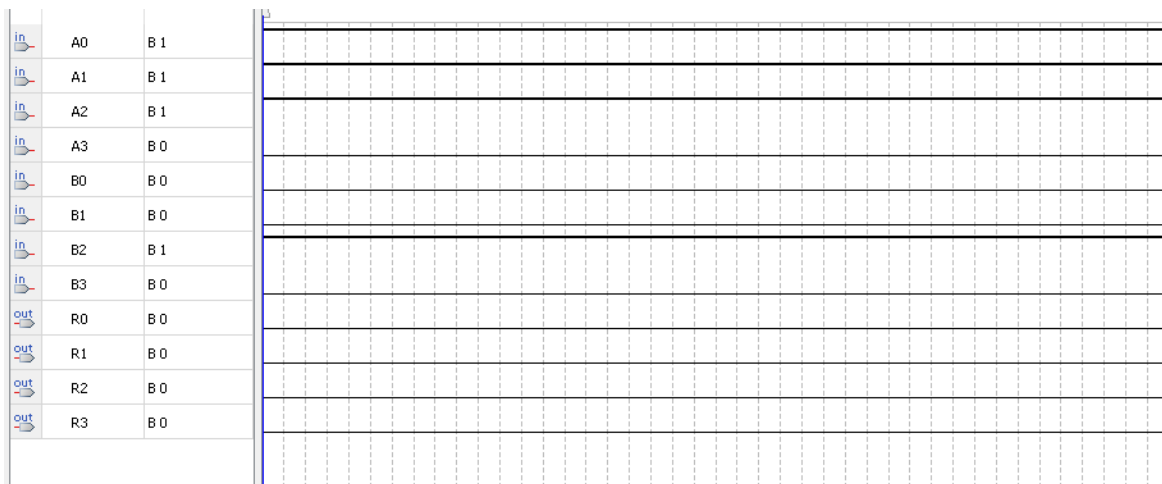
ورودی: $A = 0111 (7)$, $B = 0011 (3)$
محاسبه: $(7 + 3) \bmod 11 = 10$
خروجی مورد انتظار: $R = 1010 (10)$



شکل ۳۲: شبیه‌سازی Fax4_MOD_11 برای $3 + 7$

۳-۶-۳-۶ تست $7 + 4$

ورودی: $A = 0100 (4)$, $B = 0111 (7)$
محاسبه: $(4 + 7) \bmod 11 = 0$
خروجی مورد انتظار: $R = 0000 (0)$



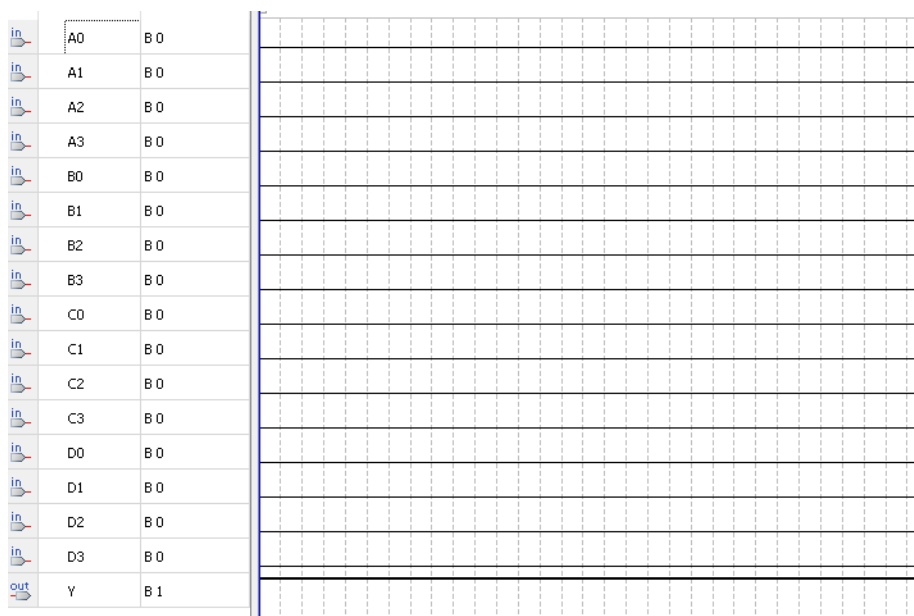
شکل ۳۳: شبیه‌سازی Fax4_MOD_11 برای ۴ + ۷

این آخری دقیقاً همان حالتی است که مسیر اصلاحیه فعال می‌شود: جمع خام برابر ۱۰۱۱ است و چون S_3 و S_1 و S_0 روشن هستند سیگنال C' یک می‌شود و ۵ به آن اضافه می‌شود تا خروجی ۰۰۰۰ شود.

۴-۶-۳ تست مدار کلی

۱-۴-۶-۳ تست ۰۰۰۰

ورودی: $A = ۰$ ، $B = ۰$ ، $C = ۰$ ، $D = ۰$ (عدد ۰۰۰۰)
محاسبه: مجموع علامت‌دار $۰ = ۰ - ۰ + ۰ - ۰$ است که بر ۱۱ بخش‌پذیر است
خروجی مورد انتظار: $Y = ۱$



شکل ۳۴: شبیه‌سازی مدار بخش‌پذیری بر ۱۱ برای عدد ۰۰۰۰

۳-۴-۲ تست ۰۰۰۱

ورودی: $A = ۱$ ، $B = ۰$ ، $C = ۰$ ، $D = ۰$ (عدد ۰۰۰۱)
محاسبه: مجموع علامت‌دار $۱ - ۰ + ۰ - ۰ = ۱$ است که بر ۱۱ بخش پذیر نیست
خروجی مورد انتظار: $Y = ۰$

in	A0	B 1
in	A1	B 0
in	A2	B 0
in	A3	B 0
in	B0	B 0
in	B1	B 0
in	B2	B 0
in	B3	B 0
in	C0	B 0
in	C1	B 0
in	C2	B 0
in	C3	B 0
in	D0	B 0
in	D1	B 0
in	D2	B 0
in	D3	B 0
out	Y	B 0

شکل ۳۵: شبیه‌سازی مدار بخش‌پذیری بر ۱۱ برای عدد ۰۰۰۱

۳-۴-۳ تست ۰۰۱۱

ورودی: $A = ۱$ ، $B = ۱$ ، $C = ۰$ ، $D = ۰$ (عدد ۰۰۱۱)
محاسبه: مجموع علامت‌دار $۰ - ۱ + ۱ - ۰ = ۰$ است که بر ۱۱ بخش پذیر است
خروجی مورد انتظار: $Y = ۱$

in	A0	B 1
in	A1	B 0
in	A2	B 0
in	A3	B 0
in	B0	B 1
in	B1	B 0
in	B2	B 0
in	B3	B 0
in	C0	B 0
in	C1	B 0
in	C2	B 0
in	C3	B 0
in	D0	B 0
in	D1	B 0
in	D2	B 0
in	D3	B 0
out	Y	B 1

شکل ۳۶: شبیه‌سازی مدار بخش‌پذیری بر ۱۱ برای عدد ۰۰۱۱

۴-۴-۳ تست ۰۰۹۹

ورودی: $A = ۹, B = ۰, C = ۰, D = ۰$ (عدد ۰۰۹۹)
محاسبه: مجموع علامت‌دار $۰ = ۰ - ۹ + ۹ - ۰$ است که بر ۱۱ بخش پذیر است
خروجی مورد انتظار: $Y = ۱$

in	A0	B 1	
in	A1	B 0	
in	A2	B 0	
in	A3	B 1	
in	B0	B 1	
in	B1	B 0	
in	B2	B 0	
in	B3	B 1	
in	C0	B 0	
in	C1	B 0	
in	C2	B 0	
in	C3	B 0	
in	D0	B 0	
in	D1	B 0	
in	D2	B 0	
in	D3	B 0	
out	Y	B 1	

شکل ۳۷: شبیه سازی مدار بخش پذیری بر ۱۱ برای عدد ۰۰۹۹

۵-۴-۳ تست ۹۲۷۳

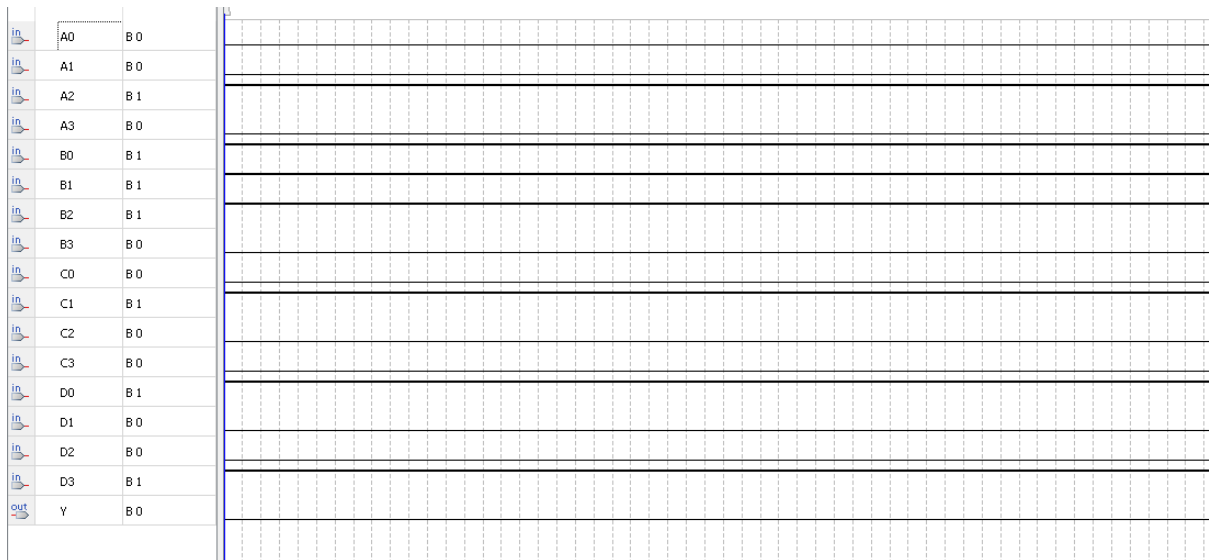
ورودی: $A = ۳, B = ۷, C = ۲, D = ۹$ (عدد ۹۲۷۳)
محاسبه: مجموع علامت‌دار $-۱۱ = ۹ - ۲ + ۷ - ۳$ است که بر ۱۱ بخش پذیر است
خروجی مورد انتظار: $Y = ۱$

in	A0	B 1	
in	A1	B 1	
in	A2	B 0	
in	A3	B 0	
in	B0	B 1	
in	B1	B 1	
in	B2	B 1	
in	B3	B 0	
in	C0	B 0	
in	C1	B 1	
in	C2	B 0	
in	C3	B 0	
in	D0	B 1	
in	D1	B 0	
in	D2	B 0	
in	D3	B 1	
out	Y	B 1	

شکل ۳۸: شبیه سازی مدار بخش پذیری بر ۱۱ برای عدد ۹۲۷۳

۹۲۷۴ تست ۳-۶-۴-۶

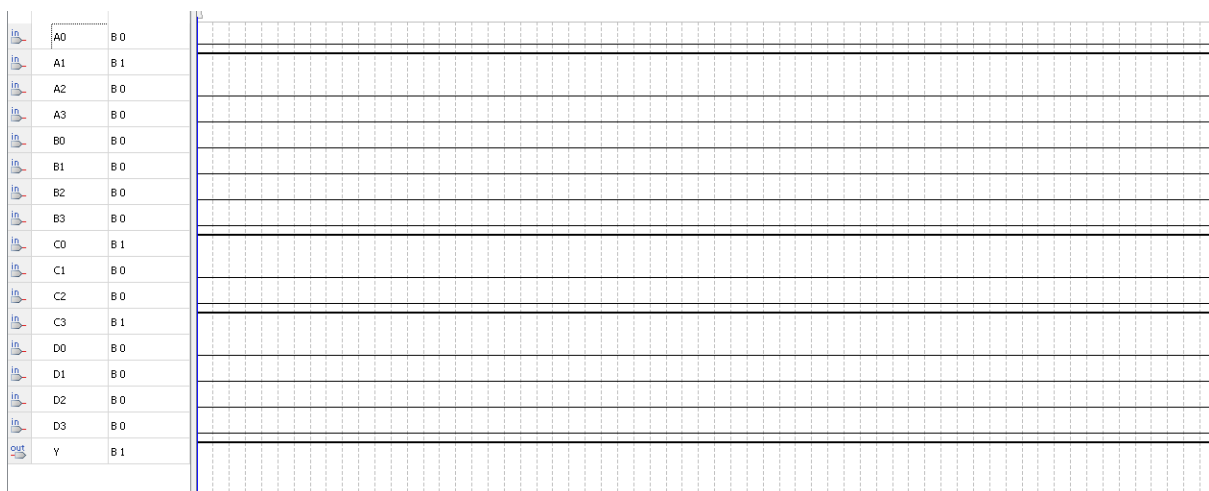
ورودی: $A = ۴$ ، $B = ۷$ ، $C = ۲$ ، $D = ۹$ (عدد ۹۲۷۴)
محاسبه: مجموع علامت‌دار $-۱۰ = ۹ - ۲ + ۷ - ۴$ است که بر ۱۱ بخش پذیر نیست
خروجی مورد انتظار: $Y = ۰$



شکل ۳۹: شبیه‌سازی مدار بخش‌پذیری بر ۱۱ برای عدد ۹۲۷۴

۰۹۰۲ تست ۳-۶-۴-۷

ورودی: $A = ۲$ ، $B = ۰$ ، $C = ۹$ ، $D = ۰$ (عدد ۰۹۰۲)
محاسبه: مجموع علامت‌دار $۱۱ = ۰ - ۰ + ۹ - ۲$ است که بر ۱۱ بخش پذیر است
خروجی مورد انتظار: $Y = ۱$



شکل ۴۰: شبیه‌سازی مدار بخش‌پذیری بر ۱۱ برای عدد ۰۹۰۲

۴ جمع‌بندی

۱. در تمام مراحل آزمایش‌ها برای اینکه از تکرار کشیدن مدار جلوگیری شود، با استفاده از سیستم طرح‌واره‌ی Quartus می‌توانیم از طرح‌واره‌های دیگر داخل مکان‌های دیگر استفاده کنیم و به این شکل از تکرار جلوگیری کنیم (شکل ۴۱).



```
..\..\..\uni/DSD_LAB/MOD_3.bdf
..\..\..\uni/DSD_LAB/Fax1.bdf
..\..\..\uni/DSD_LAB/Waveform.vwf
..\..\..\uni/DSD_LAB/Fax2_MOD_3.bdf
..\..\..\uni/DSD_LAB/main.bdf
..\..\..\uni/DSD_LAB/test_fa_mod.vwf
..\..\..\uni/DSD_LAB/test_bcd_mod.vwf
..\..\..\uni/DSD_LAB/test_main_mod3.vwf
..\..\..\uni/DSD_LAB/Fax4_MOD_11.bdf
..\..\..\uni/DSD_LAB/MOD_11_NEG.bdf
..\..\..\uni/DSD_LAB/Fax4.bdf
..\..\..\uni/DSD_LAB/test_fax4.vwf
..\..\..\uni/DSD_LAB/main2.bdf
..\..\..\uni/DSD_LAB/test_neg_mod_11.vwf
..\..\..\uni/DSD_LAB/test_fax4_mod_11.vwf
..\..\..\uni/DSD_LAB/test_main2.vwf
```

شکل ۴۱: استفاده‌ی مجدد از نماد یک طرح‌واره در طرح‌واره‌ی دیگر

۲. در هنگام آزمایش به چندین مشکل رایج برخوردیم، از جمله اینکه اسم‌های پیمانه‌ها داخل ویرایشگر^{۱۴} یکسان می‌شدند و موقع سنتز باعث تداخل می‌شدند که به سادگی با عوض کردن اسم آن‌ها قابل حل بود.

۳. در خیلی از فرمول‌ها و جدول‌های بالا مقادیر زیادی به صورت بی‌اهمیت هستند؛ دلیل وجود آن‌ها این است که ورودی در آن شرایط حالتی غیراستاندارد و یا قرارداد نشده و یا غیرممکن در مدار به خود گرفته، پس خروجی اهمیت نخواهد داشت.

۴. ساختن هر پیمانه به صورت یک طرح‌واره‌ی مستقل و ساختن نماد از آن، هم طرح را خوانا نگه می‌دارد و هم اجازه می‌دهد هر پیمانه پیش از ترکیب شدن در مدار اصلی جداگانه آزموده شود؛ همین کار باعث شد خطاهای طراحی خیلی زودتر پیدا شوند.